
Chapter 17 Binary Trees

17.1 Binary Tree Basics

- Node and Tree Classes 

17.2 Tree Traversals

- Constructing and Viewing a Tree 

17.3 Common Tree Operations

- Sum of a Tree 
- Counting Levels 
- Counting Leaves 

17.4 Binary Search Trees

- The Binary Search Tree Property 
- Building a Binary Search Tree 
- The Pattern `x = change(x)` 
- Searching the Tree 
- Binary Search Tree Complexity 

Introduction

In this chapter, we will explore a new data structure known as a binary tree. Like linked lists, binary trees are composed of interconnected nodes. But unlike linked lists, which involve a one-dimensional (straight line) sequence, binary trees can branch in two directions, which gives them a two-dimensional structure.

A surprising number of data relationships can be represented using binary trees. Any relationship that involves a division into two paths can be represented with a binary tree. Thus, binary trees can store information that involves a yes/no relationship, a high/low relationship, or a first and second relationship. For example, arithmetic expressions have operators like `+` and `*` that have a first operand and a second operand. Binary trees are a natural structure for storing such relationships. Such trees are often referred to as expression trees because they

capture the structure of the arithmetic expressions they represent.

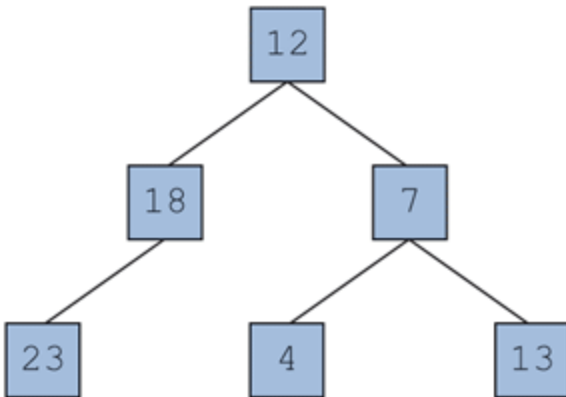
Because binary trees are linked structures, they share many of the useful properties of linked lists. It is fairly easy to grow or shrink a binary tree by rearranging the links of the nodes.

After looking at basic binary tree terminology and basic binary tree operations, we will explore a particular kind of binary tree known as a binary search tree. Binary search trees are used to capture high/low relationships and provide an efficient structure for keeping track of data that have a natural ordering. The `TreeSet` and `TreeMap` structures that are part of the collections framework are implemented as binary search trees.

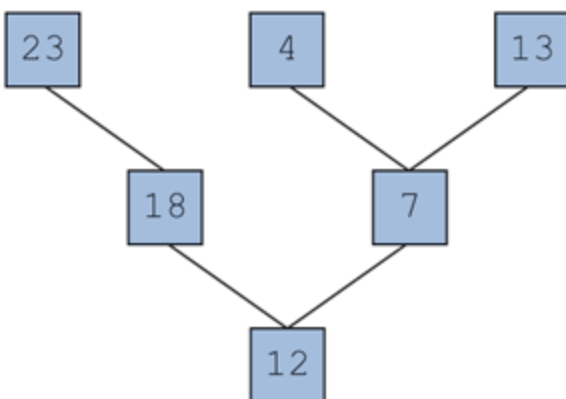
As we study binary trees, we will find that recursion comes into play quite often. Many binary tree operations are best written recursively.

17.1 Binary Tree Basics

Here is a diagram of a simple binary tree of integer values.



As we did with linked lists, we refer to each different data value as a node. This tree has a total of six nodes. Some people joke that computer scientists view the world upside down, so imagine turning the diagram around the other way:



Viewed upside down, the diagram looks more like a tree. At the bottom of this structure is a node storing the value 12. We refer to this value as the *root* of the tree. All nodes are connected either directly or indirectly to the root node. In this diagram, the nodes that store the values 12, 18, and 7 have connections branching out and up. These nodes are referred to as *branch nodes*. The nodes storing the values 23, 4, and 13 are at the top of this tree and have nothing above them. They are referred to as leaf *nodes* or *leaves* of the tree.

Before we try to formally define these terms, let's begin with a formal definition of a binary tree.

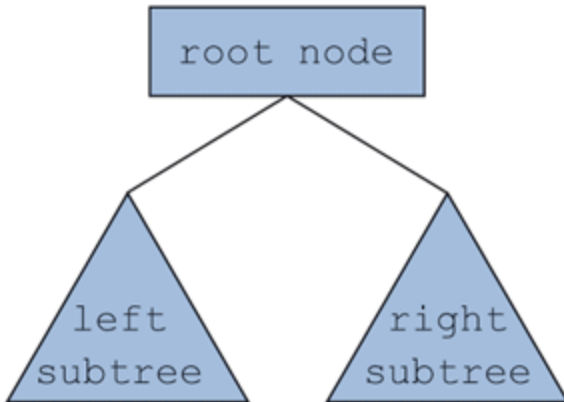
Binary Tree

A binary tree is either

- an empty tree or
- a root node (typically storing some data) that refers to two other trees known as the left subtree and the right subtree.

The key phrase in this definition is “subtree.” The tree is composed of smaller trees. This definition is recursive. The base case or simple

case is an empty tree that has no nodes at all. The recursive case describes a tree of the following form:



This recursive definition will be a useful way to think about trees as we write binary tree code. The definition will help us to more precisely define the concepts of a branch node and a leaf node.

Branch (Branch Node)

A node that has one or more nonempty subtrees.

Leaf (Leaf Node)

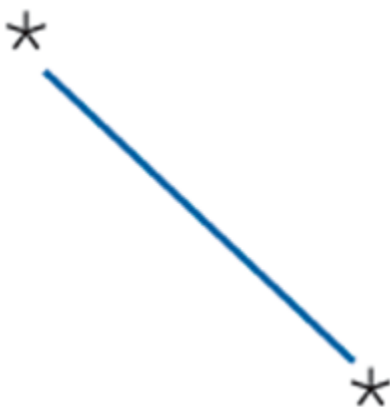
A node that has two empty subtrees.

Using our recursive definition, let's explore how we can form various kinds of trees. The simplest kind of tree is an empty tree, which can't really be drawn because it contains no values.

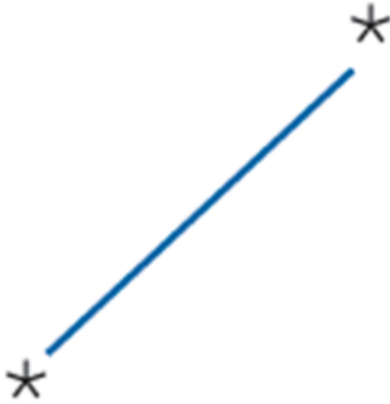
Once you understand the concept of an empty tree, you can use the second part of the definition to create a new kind of tree that is composed of a root node with left and right subtrees that are both empty. In other words, that tree would be a single leaf node, which you could represent with a star:

*

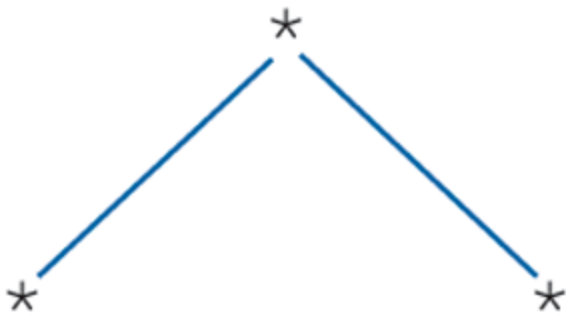
Now you can use the recursive rule to say that a tree can be a root node with a left tree that is empty and a right tree that is a leaf:



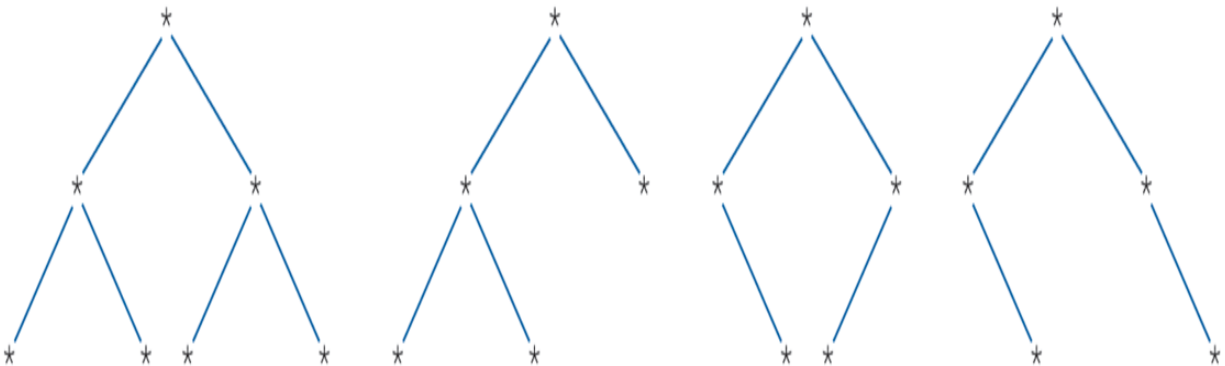
Or a tree can have an empty right and a leaf to the left:



Or a tree can have a leaf on either side:



These different forms now become possibilities to use for the recursive definition, allowing you to construct even more kinds of trees:

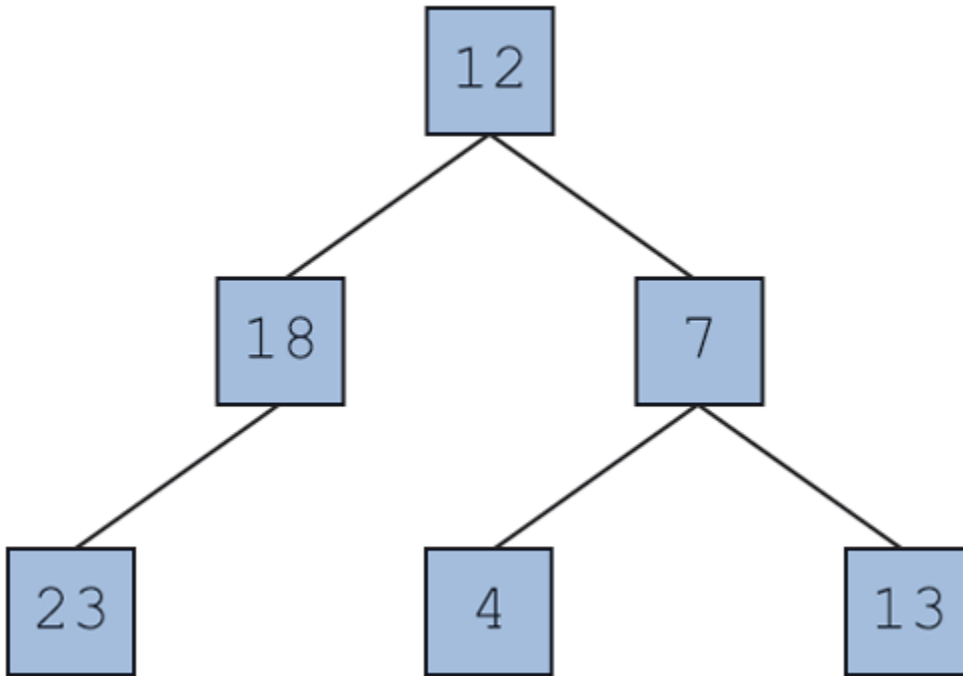


Using the recursive definition of a tree, we can define the concept of parent/child relationships in the tree:

Parent/Child/Sibling/Ancessor/Descendant

For every node p that has a nonempty subtree with root node q , we say that p is the parent of q and q is the child of p , which leads logically to ancestor relationships (parent of parent of . . .), descendant relationships (child of child of . . .), and sibling relationships (two nodes with the same parent).

Let us return to our original tree:



Let's start at the overall root node, which stores the value 12. It has two nonempty subtrees. The left subtree stores the value 18 at its root and the right subtree stores the value 7 at its root. The nodes storing 18 and 7 are children of the node storing 12 and they are siblings of each other. Similarly, the node storing 23 is a child of the node storing 18, and the nodes storing 4 and 13 are children of the node storing 7, which makes them descendants of the overall root. The overall root is the ancestor of each of the other nodes.

Using this parent/child terminology, we can more precisely define our notion of the overall root of the tree.

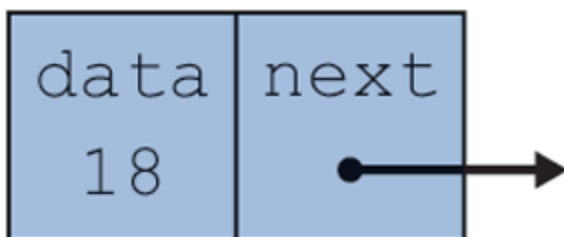
Root of a Tree (Overall Root)

The node at the top of a binary tree, the only node in the tree that has no parent.

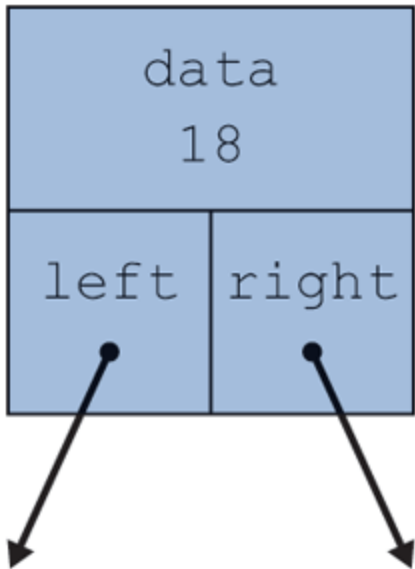
Another important aspect of binary trees is the distribution of nodes into different levels of the tree. Our sample tree has three different levels: the overall root, the children of the overall root, and the grandchildren of the overall root. This notion of levels leads us to the concept of the *depth* of an individual node (how many levels away from the root it is) and the overall *height* of the tree (how many levels it has).

Node and Tree Classes

When we studied linked lists, we found that each was built from a fairly simple building block that looks like this:



Our basic building block for a binary tree will be similar, but instead of one link, it will have two:



As we did with linked lists, we will first study how to manipulate a binary tree of simple `int` values. Here is a class definition for our binary tree nodes.

```
1 // Class for storing a single node of a binary tree of ints
2
3 public class IntTreeNode {
4     public int data;
5     public IntTreeNode left;
6     public IntTreeNode right;
7
8     // constructs a leaf node with given data
9     public IntTreeNode(int data) {
10         this(data, null, null);
11     }
12 }
```

```

11     }
12
13     // constructs a branch node with given data, left
    subtree,
14     // right subtree
15     public IntTreeNode( data, IntTreeNode left,
16                        IntTreeNode right) {
17         this.data = data;
18         this.left = left;
19         this.right = right;
20     }
21 }

```

Like our linked list node, this node is very simple; it has some public fields and a few constructors. The node has a field of type `int` for storing the data contained in this node and two links of type `IntTreeNode` for the left and right subtrees. The first constructor constructs a leaf node (using `null` for left and right). The second constructor is appropriate for a branch node where you want to specify the left and right subtrees.

The node class is not well encapsulated, but as we saw with linked lists, this is common practice in Java programming. In this chapter, we'll define a tree class that is well encapsulated and we will make sure that a client of the tree class never sees these nodes. In the final example of the chapter we will completely hide this node class

by making it a static inner class, as we did with the final version of the linked list class.

Just as we could reach every node of a linked list by starting at the front and moving forward, we can reach every node of a binary tree by starting at the overall root and moving down in the tree. As a result, we need only one field in the tree class: a reference to the root of the tree:

```
public class IntTree {  
    private IntTreeNode overallRoot;  
    ...  
}
```

We purposely use the name `overallRoot` to distinguish this root from all of the other roots. There is only one overall root. But each subtree is itself the root of a tree, and in our recursive methods we'll often use the parameter name `root` to indicate any of the roots.

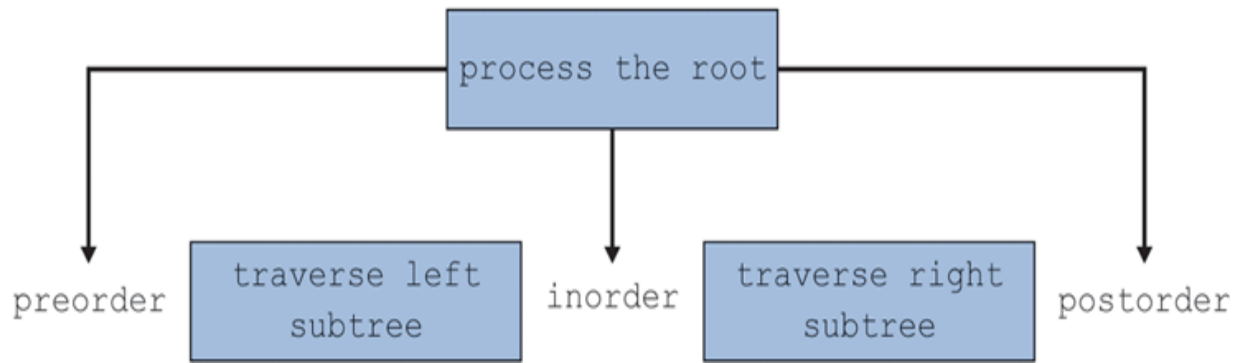
As we did with our linked lists, we'll represent the empty tree by storing the value `null` in the `overallRoot` field:

overallRoot 

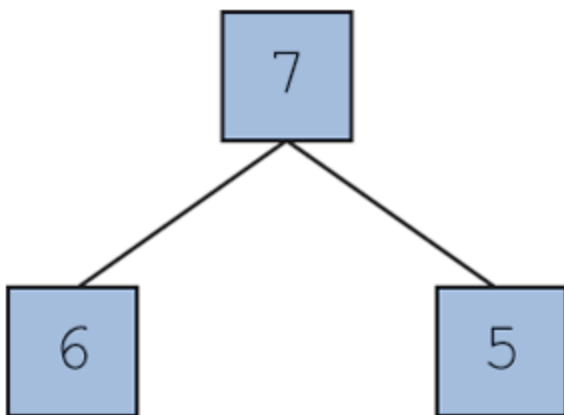
17.2 Tree Traversals

We can't do much with our tree class unless we have some way of seeing what's inside. We want to traverse the tree in such a way that we visit each node exactly once. There are many different ways to do this. Because the tree is defined recursively, it is easiest to use a recursive approach to this problem. As a result, we want to traverse the entire left subtree without dealing with any of the elements from the right and, in a separate operation, traverse the entire right subtree without dealing with any of the elements from the left. Given this decision, there are three classic binary tree traversals. Because we read from left to right, we traverse the left subtree before the right subtree. The question becomes, at what point do you deal with the root of the tree?

There are three possible answers you might give. You can process the root before you traverse either subtree, after you traverse both subtrees, or in between traversing the two subtrees. These three approaches are known as *preorder*, *postorder*, and *inorder* traversals respectively.



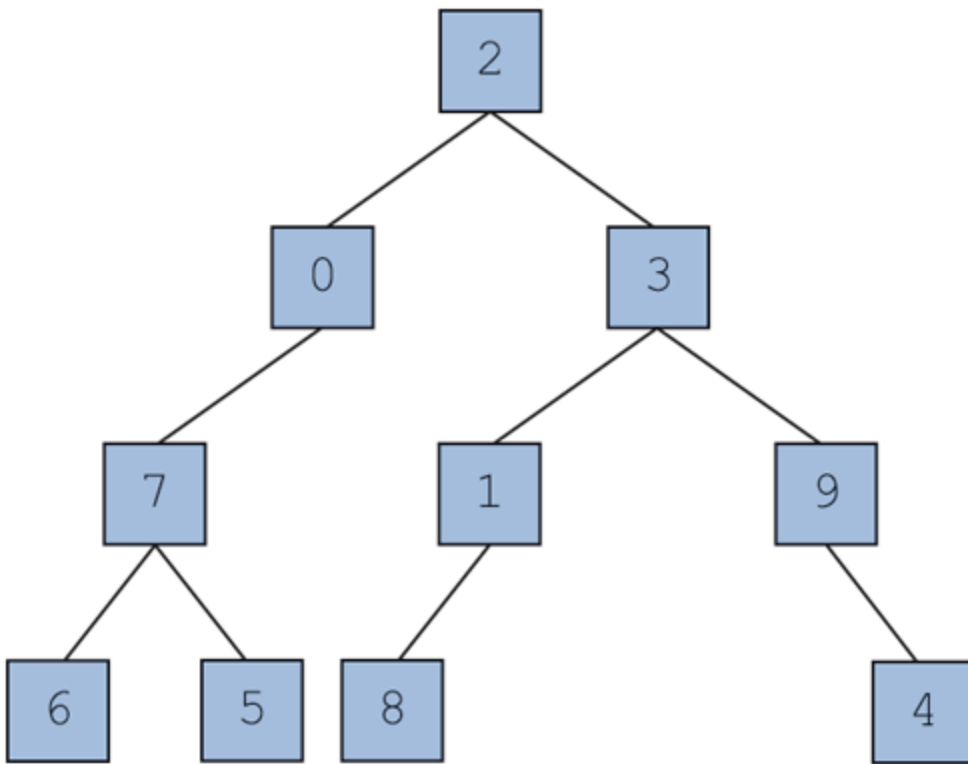
For example, consider the following simple tree:



A preorder traversal visits the root, then the left subtree, then the right subtree, yielding the sequence `[7, 6, 5]`. An inorder traversal visits the left subtree, then the root, then the right subtree, yielding the sequence `[6, 7, 5]`. A postorder traversal visits the left subtree, then the right subtree, then the root, which yields `[6, 5, 7]`. Notice in this example that we always list `6` before `5` (left before right) and that the only value which changes position is the root value `7`.

When we traverse a more complex tree, we simply use the recursive nature of this definition to carry out the traversal. For example,

consider the following tree.



The overall root stores the value `2`, so we know that the different traversals will look like this:

- Preorder traversal: `[2, <left>, <right>]`
- Inorder traversal: `[<left>, 2, <right>]`
- Postorder traversal: `[<left>, <right>, 2]`

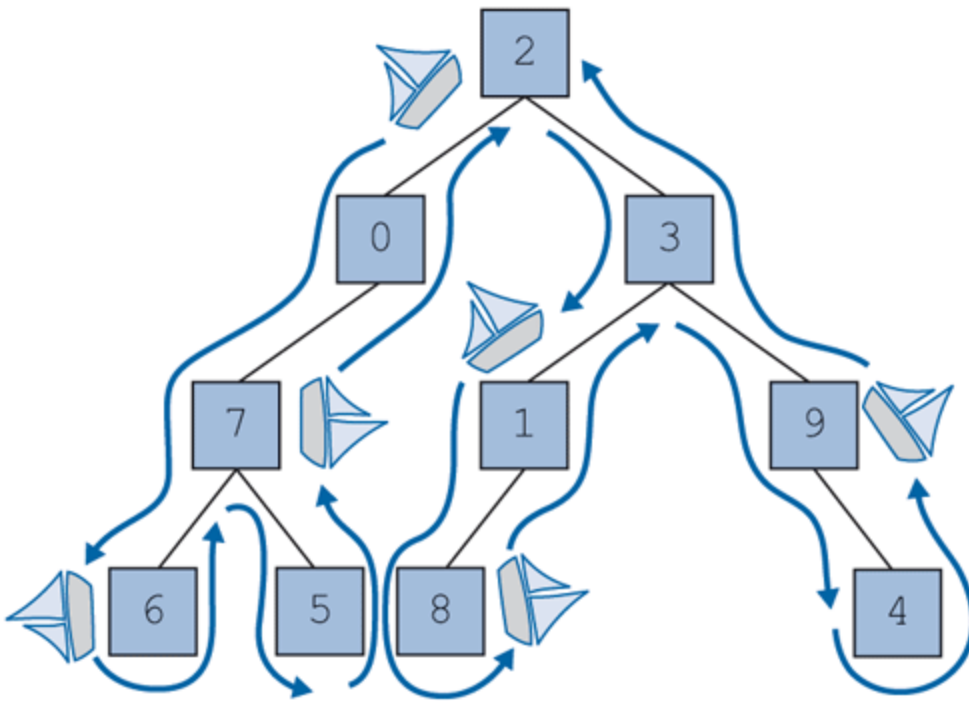
The left subtree of this tree stores the value `0` and has an empty right subtree. We know that each of the occurrences of `<left>` is going to be replaced by an appropriate traversal:

- Preorder traversal: [2, 0, <left-left>, <right>]
- Inorder traversal: [<left-left>, 0, 2, <right>]
- Postorder traversal: [<left-left>, 0, <right>, 2]

Then each occurrence of <left-left> would be replaced with an appropriate traversal for the subtree that has 7 as its root. The following are the complete traversal results:

- Preorder traversal: [2, 0, 7, 6, 5, 3, 1, 8, 9, 4]
- Inorder traversal: [6, 7, 5, 0, 2, 8, 1, 3, 9, 4]
- Postorder traversal: [6, 5, 7, 0, 8, 1, 4, 9, 3, 2]

There is another way to find the three different traversals. Imagine that each node in the tree is an island and that each link is a solid wall. Then imagine a sailboat sailing in from the upper left part of the tree and sailing around the tree, always hugging the coastline, and continuing around until it sails past the root on the right-hand side:



The sailboat sails past the root node three different times. It passes it first on the left, then underneath, and then on the right. If you think about it the right way, you'll realize that the sailboat passes each of the nodes three times. Even for a leaf node like the one on the left that stores 6, it has to pass on the left, underneath, and on the right. It does them all in a row for a leaf node, but it still passes by in all three directions.

If you print the values as the sailboat passes them on the left, you get a preorder traversal. If you print the values as the sailboat passes underneath, you get an inorder traversal. And if you print the values as the sailboat passes them on the right, you get a postorder traversal. This technique is useful for double-checking your understanding of the different traversals.

Let's now see how we can write public methods for the `IntTree` class that will perform these traversals. Let's begin by writing a public method that will print the preorder traversal of a tree. Your public method will look like this:

```
public void printPreorder() {  
    ...  
}
```

At this point we run into a problem that we discussed in [Chapter 12](#). Often when you write a recursive solution, you have to introduce a second method that we call a helper method. When you do so, it is best to write the second method as a private method. For these binary tree methods, it will almost always be the case that you actually have to write a pair of methods to solve any of these problems.

The issue is that the client makes a request to print the entire tree. But to solve this problem recursively, you need a method that works on every subtree, not just the overall tree. In other words, you need a method that takes an `IntTreeNode` as a parameter so that the recursive method will match the recursive structure of the tree. That way each recursive call will be able to pass on smaller and smaller subtrees, which allows you to get closer to reaching a base case. As a result, you will see a common pattern in binary tree code that looks like this:

```

public void doSomething(<parameters>) {
    doSomething(overallRoot, <parameters>);
}

private void doSomething(IntTreeNode root, <parameters>) {
    ...
}

```

Binary tree code won't always follow this pattern exactly because there might be other actions to perform in the public method, the return type might not be `void`, or there might be other parameters. But this general pattern will be there even with all of these variations.

To solve the problem of printing in preorder, you need a private method that has an `IntTreeNode` parameter:

```

private void printPreorder(IntTreeNode root) {
    ...
}

```

You start the recursive process by passing it the overall root. You can also include code to print text at the beginning and end of the line of output that you want to produce:

```

public void printPreorder() {
    System.out.print("preorder:");
}

```

```
printPreorder(overallRoot);  
System.out.println();  
}
```

How do you write the private method? It's good to go back to the basic definition of a binary tree. Remember that it is either an empty tree or a root node with left and right subtrees. If it's an empty tree, then there isn't anything to print. That means you could begin your private method this way:

```
private void printPreorder(IntTreeNode root) {  
    if (root == null) {  
        // do nothing  
    }  
    ...  
}
```

But since this case leaves the method with nothing to do, it's better to test the negation of this statement:

```
private void printPreorder(IntTreeNode root) {  
    if (root != null) {  
        ...  
    }  
}
```

And what do you do if `root` is not `null`? In that case, you have a root node with some data in it and you have left and right subtrees that need to be printed. In a preorder traversal you handle the root node first, which means you'd print out the data for the root. You can include a space to separate it from other values on the line of output:

```
private void printPreorder(IntTreeNode root) {  
    if (root != null) {  
        System.out.print(" " + root.data);  
        ...  
    }  
}
```

What do you do after you print the data for this node? You want to print the left subtree in a preorder manner and then print the right subtree in a preorder manner:

```
private void printPreorder(IntTreeNode root) {  
    if (root != null) {  
        System.out.print(" " + root.data);  
        // then print left subtree in a preorder manner  
        // then print right subtree in a preorder manner  
    }  
}
```

This is a time to make the leap of faith that is so essential in writing recursive methods. Thinking recursively, you'll think, "If only I had a method to print a subtree in a preorder manner . . . but I do have such a method . . . the one I'm writing." Therefore, this method becomes:

```
private void printPreorder(IntTreeNode root) {  
    if (root != null) {  
        System.out.print(" " + root.data);  
        printPreorder(root.left);  
        printPreorder(root.right);  
    }  
}
```

That modification completes the method. It may seem odd that it takes so little code to perform this task, but as we saw in [Chapter 12](#), recursive code often ends up being very short.

Now let's consider how to modify the code to print the tree in an inorder manner. The new code will have a similar public/private pair of methods:

```
public void printInorder() {  
    System.out.print("inorder:");  
    printInorder(overallRoot);  
    System.out.println();  
}
```

```
}  
  
private void printInorder(IntTreeNode root) {  
    ...  
}
```

The code will also have a similar test for an empty tree. But how do you change the body of the private printing method to make it print using an inorder traversal rather than a preorder traversal? You put the `print` in between the two recursive calls:

```
private void printInorder(IntTreeNode root) {  
    if (root != null) {  
        printInorder(root.left);  
        System.out.print(" " + root.data);  
        printInorder(root.right);  
    }  
}
```

This simple rearrangement of the three calls causes the code to print in an inorder manner rather than a preorder manner.

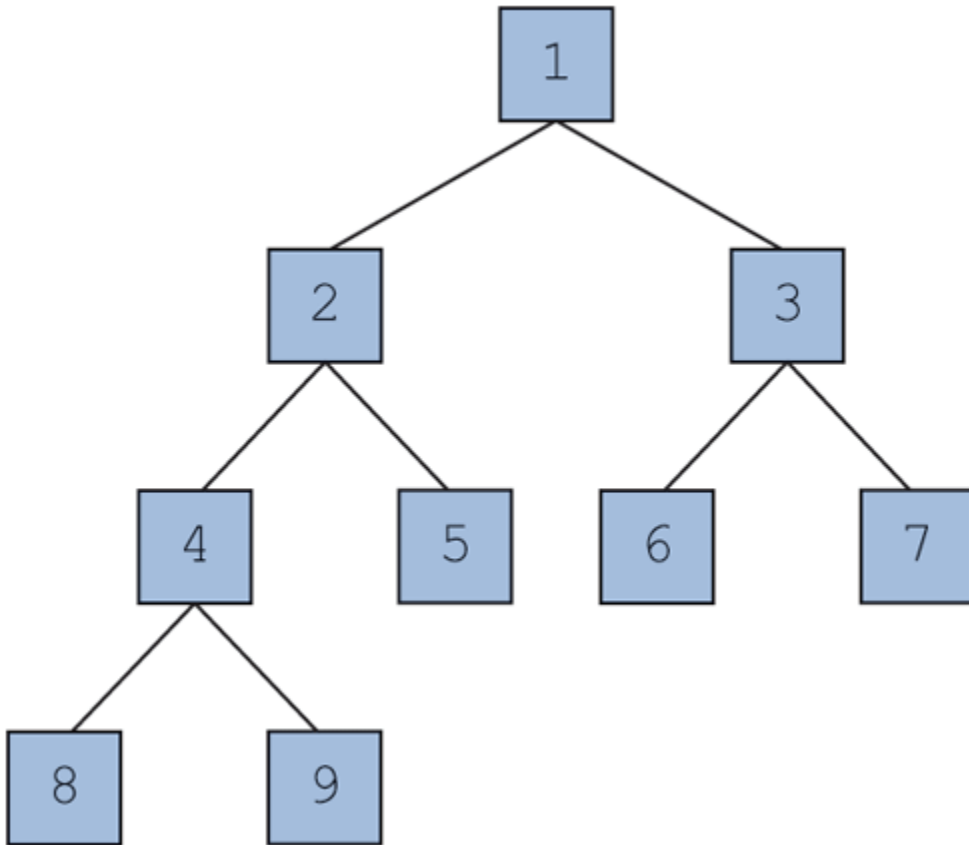
You can create another set of methods for printing in postorder by again rearranging the three operations so that the two recursive calls are made before the `print`.

It is important to understand why this seemingly small change of moving the `print` statement relative to the recursive calls generates such different behavior. It's because each recursive call potentially processes a large amount of data (an entire subtree). You'll find that this is a common property of binary tree code. A minor change to the code can produce very different results.

Constructing and Viewing a Tree

It's difficult to know whether our traversal methods work unless we can construct a tree and examine its structure. Most often we develop binary tree code with a specific application in mind. We're trying to keep things simple initially, so for now, let's construct a very specific tree that will allow us to test our code.

Using recursion, we can fairly easily construct a tree that has nodes numbered in a sequential manner level by level, as in this tree of nine nodes:



Notice how the number 1 is on the first level, then 2 and 3 on the next level, then 4, 5, 6, and 7, and so on. In this structure, we will fill in every possible node up to some maximum node number. Let's refer to this as a *sequential tree* to simplify our discussion.

Assume that the constructor for the class will be passed the maximum node number to include in the tree:

```
public IntTree(int max) {  
    ...  
}
```

This is another case in which you need to introduce a private method to do most of the work for you. In our previous pattern, we wrote methods that take a parameter of type `IntTreeNode` and our initial call passed the value of the field `overallRoot` to start the ball rolling. In this case, there is no tree to traverse. The whole point of the method is to construct the tree. Instead of passing `overallRoot` as a parameter, we want to use the method to give a value to `overallRoot`. And instead of passing a parameter of type `IntTreeNode`, we want the method to return a value of type `IntTreeNode` (a reference to the tree it has built). Thus, the pair of methods will look something like this:

```
public IntTree(int max) {  
    overallRoot = buildTree();  
}  
  
private IntTreeNode buildTree() {  
    ...  
}
```

This is a good first approximation, but it isn't right. As you learn to write code for binary trees, you will discover that one of the most difficult tasks is figuring out the appropriate parameters to pass. As you practice more, you will see various patterns emerge.

One obvious parameter to include here is the value `max` that is passed to the public method. How can the private method do its job properly if it doesn't know when to stop? You can modify your pair to include that parameter:

```
public IntTree(int max) {  
    overallRoot = buildTree(max);  
}  
  
private IntTreeNode buildTree(int max) {  
    ...  
}
```

This still isn't enough information for the private method to do its job. Think of it this way: The private method has to construct each of the different subtrees that are involved. One call on the method will construct the overall tree. Another will construct the left subtree of the overall root. Another will construct the right subtree of the overall root. How is the method supposed to know which of these tasks to solve?

We must include an extra parameter telling the method the node number to use for the root of the tree that it is constructing. When the method is passed the value `1`, it will construct the overall root. When it is passed the value `2`, it will construct the left subtree of the overall root, and so on. Including this second parameter, we end up with the following code:

```

public IntTree(int max) {
    overallRoot = buildTree(1, max);
}

// returns a sequential tree with n as its root unless n is
// greater than max, in which case it returns an empty tree
private IntTreeNode buildTree(int n, int max) {
    ...
}

```

Notice that in the public method you need to pass the value `1` to indicate that the first call should be generating a tree with `1` as its root value. And as the comments indicate, you only want to construct the tree if the value of `n` is less than or equal to the value of `max`.

The requirement that you want to stop building the tree for values of `n` that are greater than `max` gives you an appropriate base case for the private method:

```

private IntTreeNode buildTree(int n, int max) {
    if (n > max) {
        return null;
    } else {
        ...
    }
}

```

Now you just have to fill in the recursive case. You know that you need to construct a node that stores the value `n`. The less obvious part is how to construct the left and right subtrees. For the moment, you can introduce some local variables for the two subtrees; later, you will fill in the details about how to construct them. The method so far looks like this:

```
private IntTreeNode buildTree(int n, int max) {  
    if (n > max) {  
        return null;  
    } else {  
        IntTreeNode left = ...  
        IntTreeNode right = ...  
        return new IntTreeNode(n, left, right);  
    }  
}
```

We suggested building this particular tree because there is a simple mathematical relationship between each node in this tree and its children. If a node is numbered n , then its left child is numbered $2n$ and its right child is numbered $2n + 1$. For example, the overall root is numbered `1` and its children are numbered `2` and `3`. The left subtree is numbered `2` and its children are numbered `4` and `5`, and so on.

Getting back to our code, now that we know what value to store in each of the two children, we can use recursive calls to construct the left and right subtrees:

```
private IntTreeNode buildTree(int n, int max) {  
    if (n > max) {  
        return null;  
    } else {  
        IntTreeNode left = buildTree(2 * n, max);  
        IntTreeNode right = buildTree(2 * n + 1, max);  
        return new IntTreeNode(n, left, right);  
    }  
}
```

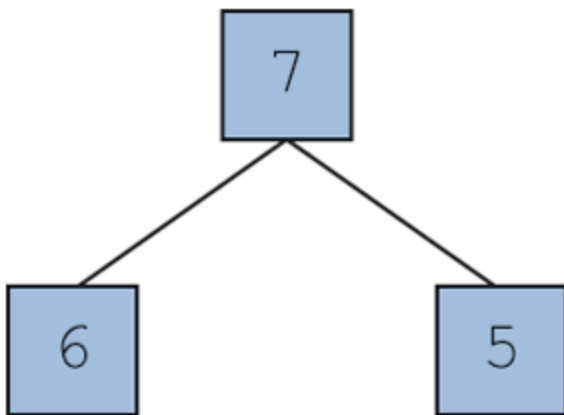
The preceding code works and is very clear in terms of the steps performed. In the recursive case (the `else` branch), you can see that you construct a left subtree, then construct a right subtree, and finally put these pieces together into a new `IntTreeNode` that is returned by the method. Although this code is clear, many people prefer to write it even more concisely to eliminate the local variables. You can rewrite the method above as follows:

```
private IntTreeNode buildTree(int n, int max) {  
    if (n > max) {  
        return null;  
    } else {
```

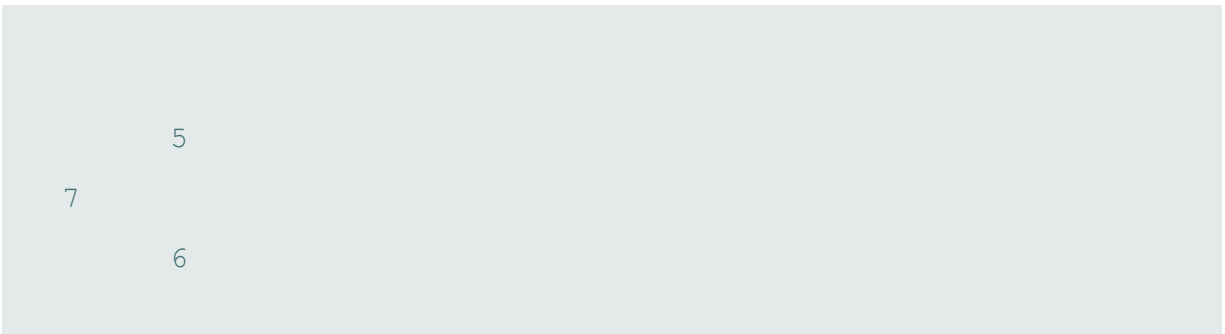
```
return new IntTreeNode(n, buildTree(2 * n, max),  
                        buildTree(2 * n + 1, max));  
}  
}
```

Even though this tree ends up having a very predictable structure, it is helpful to develop a method that shows the structure more clearly. Some programming environments like jGRASP have “structure viewers” that allow you to see a visual representation of a binary tree. If you don’t have access to such a program, you might want to take a simplified approach to viewing the structure of the tree.

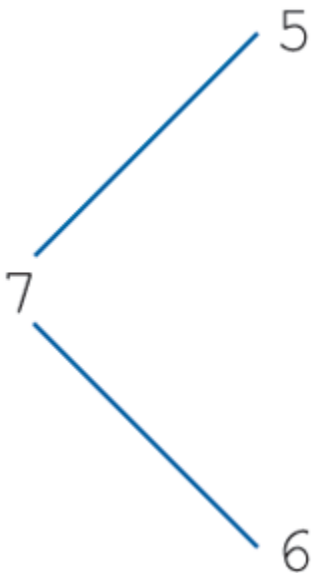
You can accomplish this by writing a simple variation of your inorder traversal. Instead of printing values all on a line, you will print them one per line and use indentation to indicate the level of the tree. For example, consider the following simple tree:



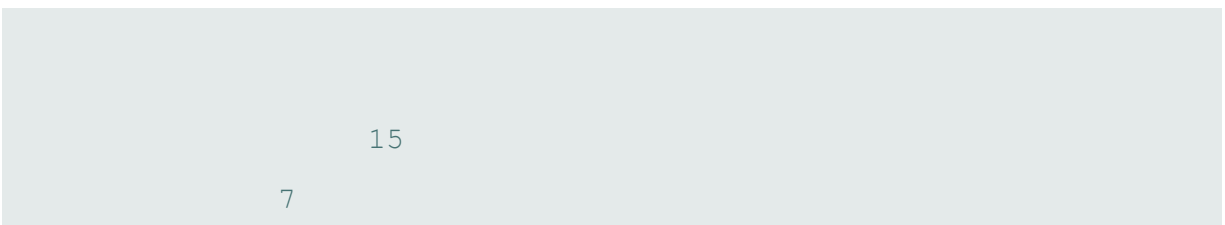
You’d want your program to produce output like this:

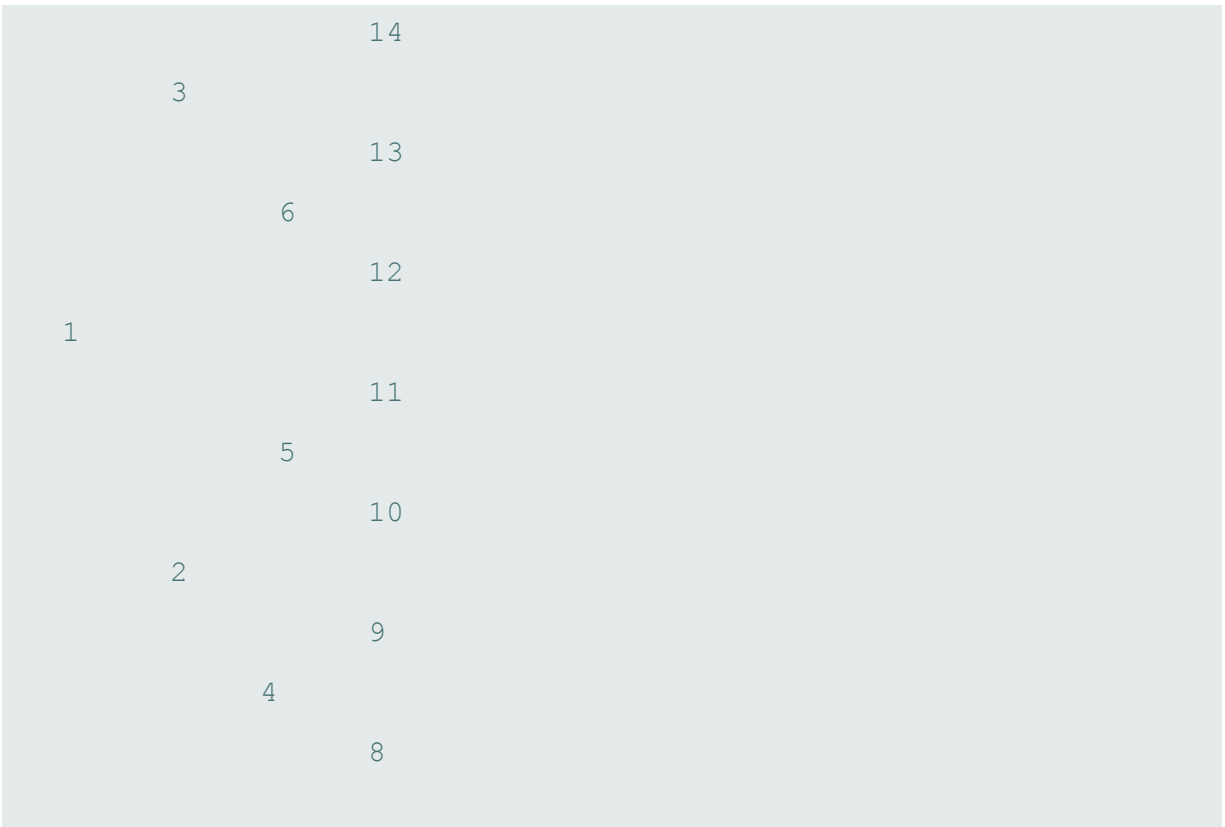


To read this output, you have to imagine rotating the page 90 degrees in a clockwise fashion or tilting your head slightly to the left. The output is a little easier to see if we throw in some lines:

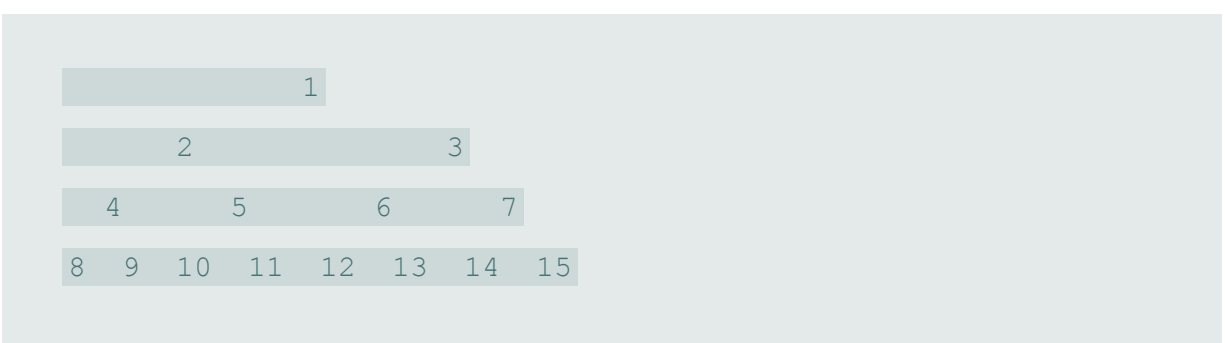


The lines are a little tricky to produce, so let's settle for producing the simple output and you'll have to mentally add the lines. For example, you might get output like the following for a tree with four levels:

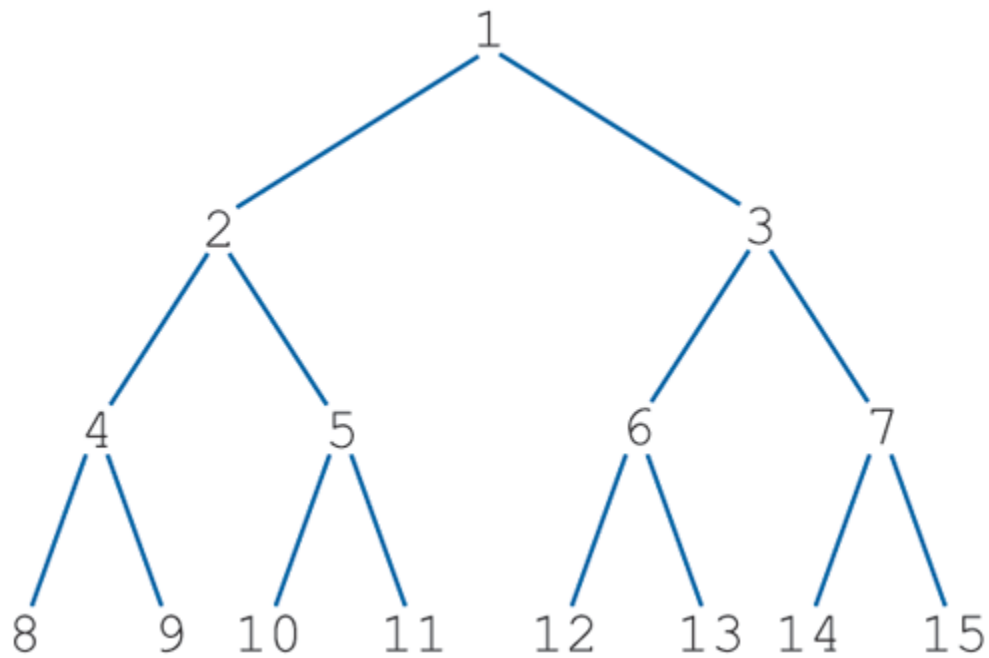




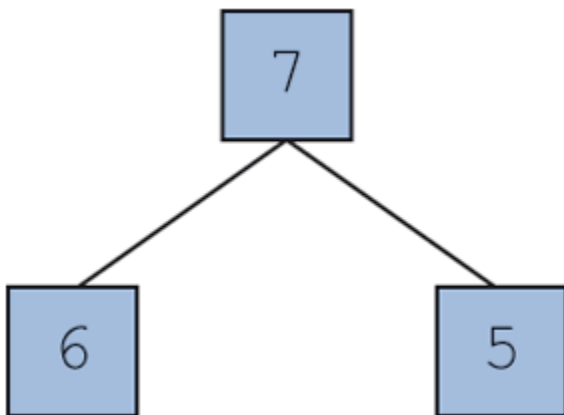
Again, imagine rotating this output 90 degrees (or rotating the page on which it is printed). The result looks like this:



If you add lines to this diagram, you'll see the tree:



The output has the root node between its two subtrees, so this traversal is a variation of the inorder traversal. Oddly enough, you want to print the right subtree before the left subtree. For example, consider the following tree:



Your method would produce the following output:

```
graph TD; 5 --> 7; 5 --> 6;
```

Notice that the right subtree value of **5** is printed first, then the root, then the left subtree value of **6**. This happens because you are rotating the image.

You can fairly easily modify your inorder traversal code to produce this new set of methods:

```
public void printSideways() {  
    printSideways(overallRoot);  
}  
  
private void printSideways(IntTreeNode root) {  
    if (root != null) {  
        printSideways(root.right);  
        System.out.println(root.data);  
        printSideways(root.left);  
    }  
}
```

But this version of the code does not indent the various levels of the tree. To obtain the proper indentation, you need to introduce an extra

parameter to let the private method know the level at which each node appears. The initial call on the private method should pass the value `0` because the overall root should be indented `0` spaces. Each recursive call should pass an indentation level that is one higher for its children. The following code produces the indentation of the tree:

```
public void printSideways() {  
    printSideways(overallRoot, 0);  
}  
  
private void printSideways(IntTreeNode root, int level) {  
    if (root != null) {  
        printSideways(root.right, level + 1);  
        for (int i = 0; i < level; i++) {  
            System.out.print("    ");  
        }  
        System.out.println(root.data);  
        printSideways(root.left, level + 1);  
    }  
}
```

Here is some client code that calls the various methods we have developed:

```
1 // Short program that demonstrates the use of the IntTree  
class.
```

```

2
3 public class IntTreeClient {
4     public static void main(String[] args) {
5         IntTree t = new IntTree(12);
6         System.out.println("Tree structure:");
7         t.printSideways();
8         System.out.println();
9         t.printPreorder();
10        t.printInorder();
11        t.printPostorder();
12    }
13 }

```

This method produces the following output:

Tree structure:

```

      7
     3
      6
       12
1    11
     5
      10
     2
      9

```

```

        4
      2  8
     1 3 5 7
preorder: 1 2 4 8 9 5 10 11 3 6 12 7
inorder:  8 4 9 2 10 5 11 1 12 6 3 7
postorder: 8 9 4 10 11 5 2 12 6 7 3 1

```

Here is the complete `IntTree` class:

```

1  // Simple binary tree class that includes methods to
construct a
2  // tree of ints, to print the structure, and to print the
data
3  // using a preorder, inorder, or postorder traversal. The
trees
4  // built have nodes numbered starting with 1 and numbered
5  // sequentially level by level with no gaps in the tree.
The
6  // documentation refers to these as "sequential trees."
7
8  public class IntTree {
9      private IntTreeNode overallRoot;
10
11      // pre : max >= 0 (throws IllegalArgumentException if
not)
12      // post: constructs a sequential tree with given number
of
13      //      nodes

```

```

14     public IntTree(int max) {
15         if (max < 0) {
16             throw new IllegalArgumentException("max: " +
max);
17         }
18         overallRoot = buildTree(1, max);
19     }
20
21     // post: returns a sequential tree with n as its root
unless
22     //      n is greater than max, in which case it returns
an
23     //      empty tree
24     private IntTreeNode buildTree(int n, int max) {
25         if (n > max) {
26             return null;
27         } else {
28             return new IntTreeNode(n, buildTree(2 * n,
max),
29                                     buildTree(2 * n + 1,
max));
30         }
31     }
32
33     // post: prints the tree contents using a preorder
traversal
34     public void printPreorder() {
35         System.out.print("preorder:");

```

```
36         printPreorder(overallRoot);
37         System.out.println();
38     }
39
40     // post: prints in preorder the tree with given root
41     private void printPreorder(IntTreeNode root) {
42         if (root != null) {
43             System.out.print(" " + root.data);
44             printPreorder(root.left);
45             printPreorder(root.right);
46         }
47     }
48
49     // post: prints the tree contents using an inorder
traversal
50     public void printInorder() {
51         System.out.print("inorder:");
52         printInorder(overallRoot);
53         System.out.println();
54     }
55
56     // post: prints in inorder the tree with given root
57     private void printInorder(IntTreeNode root) {
58         if (root != null) {
59             printInorder(root.left);
60             System.out.print(" " + root.data);
61             printInorder(root.right);
62         }
```

```

63     }
64
65     // post: prints the tree contents using a postorder
traversal
66     public void printPostorder() {
67         System.out.print("postorder:");
68         printPostorder(overallRoot);
69         System.out.println();
70     }
71
72     // post: prints in postorder the tree with given root
73     private void printPostorder(IntTreeNode root) {
74         if (root != null) {
75             printPostorder(root.left);
76             printPostorder(root.right);
77             System.out.print(" " + root.data);
78         }
79     }
80
81     // post: prints the tree contents, one per line,
following an
82     //         inorder traversal and using indentation to
indicate
83     //         node depth; prints right to left so that it
looks
84     //         correct when the output is rotated.
85     public void printSideways() {
86         printSideways(overallRoot, 0);

```

```
87     }
88
89     // post: prints in reversed preorder the tree with
given
90     //     root, indenting each line to the given level
91     private void printSideways(IntTreeNode root, int level)
{
92         if (root != null) {
93             printSideways(root.right, level + 1);
94             for (int i = 0; i < level; i++) {
95                 System.out.print("    ");
96             }
97             System.out.println(root.data);
98             printSideways(root.left, level + 1);
99         }
100     }
101 }
```

17.3 Common Tree Operations



This section discusses several common binary tree operations. All of these operations are built on top of a standard tree traversal and involve reasoning recursively about the subtrees.

Sum of a Tree

First let's write a method to find the sum of the values stored in a binary tree of `ints`. The method header will look like this:

```
public int sum() {  
    ...  
}
```

As usual, you will need to introduce a private method that allows you to pass a node as a parameter so that you can specify which subtree of the overall tree to work with. The public method should call the private method, pass it the overall root, and return the result:

```
public int sum() {  
    return sum(overallRoot);  
}  
  
private int sum(IntTreeNode root) {  
    ...  
}
```

The recursive definition tells us that a tree is either empty or a root node with left and right subtrees. This definition provides an excellent basis for recursive code. You know that the empty tree has a sum of 0, so you can begin with that case:

```
private int sum(IntTreeNode root) {  
    if (root == null) {  
        return 0;  
    } else {  
        ...  
    }  
}
```

If the tree is not empty, then it is a root node that has left and right subtrees. To find the sum of the values, you have to combine all three parts: the data stored in the root node and the sums of the left and right subtrees. This task translates very easily into recursive code:

```
private int sum(IntTreeNode root) {  
    if (root == null) {  
        return 0;  
    } else {  
        return root.data + sum(root.left) + sum(root.right);  
    }  
}
```

Counting Levels



As a second example, let's write a method called `countLevels` that returns the number of levels in a tree. For our purposes, we consider the root node to be at level `1`, its children to be at level `2`, its grandchildren to be at level `3`, and so on. The `countLevels` method should return the level of the node that is furthest from the root.

You can again solve this task by writing a public/private pair of methods. The public method returns the result of invoking the private method with the overall root:

```
public int countLevels() {  
    return countLevels(overallRoot);  
}  
  
private int countLevels(IntTreeNode root) {  
    ...  
}
```

You can again use the definition that a tree is either empty or a root node with left and right subtrees. If it is empty, then it has no levels at all (0 levels):

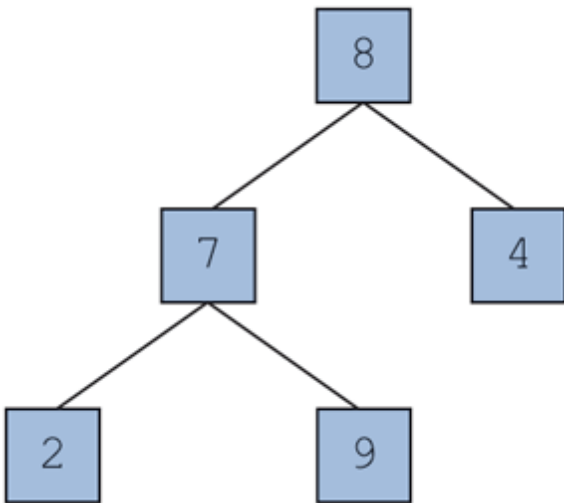
```
private int countLevels(IntTreeNode root) {  
    if (root == null) {  
        return 0;  
    } else {  
        ...  
    }  
}
```

```
}  
}
```

If the tree is not empty, then it is a root node with left and right subtrees. In this case, the data stored in the tree don't matter. We are asking a question about the structure of the tree. To solve this problem, you should think about what a recursive call on the subtrees would return. It would tell you the number of levels in the left subtree and the number of levels in the right subtree. Those answers might match (they might have the same number of levels), but often you'll find that one subtree has more levels than the other. In that case, what matters more is the subtree that has more levels, because that determines the number of levels in the overall tree. Here's a first attempt at writing the method:

```
private int countLevels(IntTreeNode root) {  
    if (root == null) {  
        return 0;  
    } else {  
        return Math.max(countLevels(root.left),  
countLevels(root.right));  
    }  
}
```

Let's think about a specific case. Consider the following tree:



If the method has been called with the overall root of the tree as a parameter, then you expect the recursive calls to return `2` as the number of levels for the left subtree and `1` as the number of levels for the right subtree. The call on `Math.max` will correctly choose the `2` over the `1` for the overall answer, but the overall tree doesn't have two levels. It has three levels. You have to account for the root node, which is an extra level, by adding `1` to the previous expression:

```
private int countLevels(IntTreeNode root) {  
    if (root == null) {  
        return 0;  
    } else {  
        return 1 + Math.max(countLevels(root.left),  
countLevels(root.right));  
    }  
}
```

Without the “1 +” in this expression, the method would always return an answer of 0. This is another example of a case in which a minor change in a recursive definition makes a big difference in how the method behaves.

The *height* of a binary tree is 1 less than its number of levels. When we count levels, we say that the empty tree has 0 levels and a tree composed of a single node has 1 level. According to the standard definition of binary tree height, a tree of one node is considered to have a height of 0 and the empty tree is considered either to have no height (undefined height) or a height of -1.

Counting Leaves

As a final example, let's write a method that returns a count of the number of leaf nodes in a tree. It will have the familiar public/private pair of methods that includes an initial call with the overall root. You can once again use the recursive definition of a tree to help you write this code. If a tree is empty, then it has no nodes at all, let alone any leaf nodes. Thus, you can begin your method with the following lines of code:

```
public int countLeaves() {  
    return countLeaves(overallRoot);  
}
```

```
private int countLeaves(IntTreeNode root) {  
    if (root == null) {  
        return 0;  
    } else {  
        ...  
    }  
}
```

The `else` case involves a root node with left and right subtrees. How many leaf nodes are in such a tree? Think about what recursive calls on the left and right subtrees will return. They will tell you how many leaves are in the subtrees. Each of those leaves is a leaf of the overall tree, so you might think that the answer is as simple as returning the sum of the subtree leaves:

```
private int countLeaves(IntTreeNode root) {  
    if (root == null) {  
        return 0;  
    } else {  
        return countLeaves(root.left) + countLeaves(root.right);  
    }  
}
```

If you were to test this version of the code, you'd find that it always returns `0` as the number of leaves in a tree. That's because you

forgot one important case. When you have a root node that has left and right subtrees, the number of leaves in that tree is generally the sum of the number of leaves in the two subtrees, except in one important case. What if the subtrees are both empty? That would mean that the node you are looking at is itself a leaf node. That particular tree has exactly one leaf node (the root). You have to introduce a special case that handles that particular situation:

```
private int countLeaves(IntTreeNode root) {  
    if (root == null) {  
        return 0;  
    } else if (root.left == null && root.right == null) {  
        return 1;  
    } else {  
        return countLeaves(root.left) + countLeaves(root.right);  
    }  
}
```

It may seem odd, but that simple change makes this code work. Each leaf node returns an answer of 1, and those answers are added together by the other calls to produce a count of the various leaf nodes in the tree.

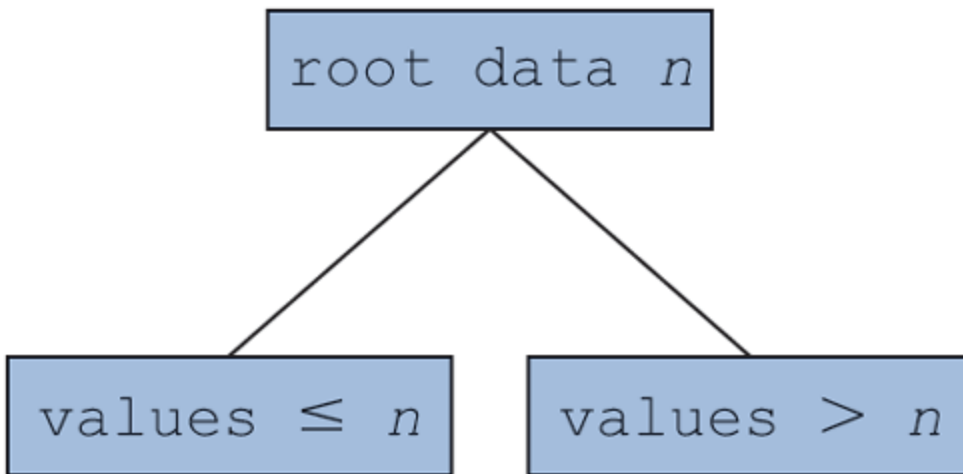
17.4 Binary Search Trees

In this section, we will study a particularly useful kind of binary tree known as a *binary search tree*. Recall from [Chapter 13](#)  that we can efficiently search through a sorted array using the binary search algorithm. That algorithm is highly efficient because on each iteration it divides a range of sorted values in half, which allows you to eliminate half of the possibilities.

A binary search tree is a structural parallel to the binary search algorithm. At each level of the tree we divide the data in half, storing approximately half the values in the left subtree and approximately half the values in the right subtree. The result is a highly efficient structure for storing a sorted sequence of values that can be quickly searched for particular values. As we noted in the introduction, both the `TreeMap` and `TreeSet` classes in the Java Collections Framework are binary search trees. In this section, we will explore how to add values to a binary search tree and how to search the tree. We will then explore some of the complexity issues related to binary search trees. We'll see that binary search trees can be very efficient, although they also can lose their efficiency if they become unbalanced. If you continue to study computer science, you will learn various techniques which guarantee that the tree stays balanced.

The Binary Search Tree Property

The values in a binary search tree are guaranteed to be arranged in such a way that the values in the left subtree are all less than the root data and the values in the right subtree are all greater than the root data. Sometimes we want to allow duplicates, in which case we have to adopt a convention about which subtree they appear in. It doesn't matter what convention we adopt, as long as we are consistent. For example, we can decide that if duplicates are allowed, then they should appear in the left subtree, which would lead to the following property:



This property of nodes is known as the *binary search tree property*.

Binary Search Tree Property

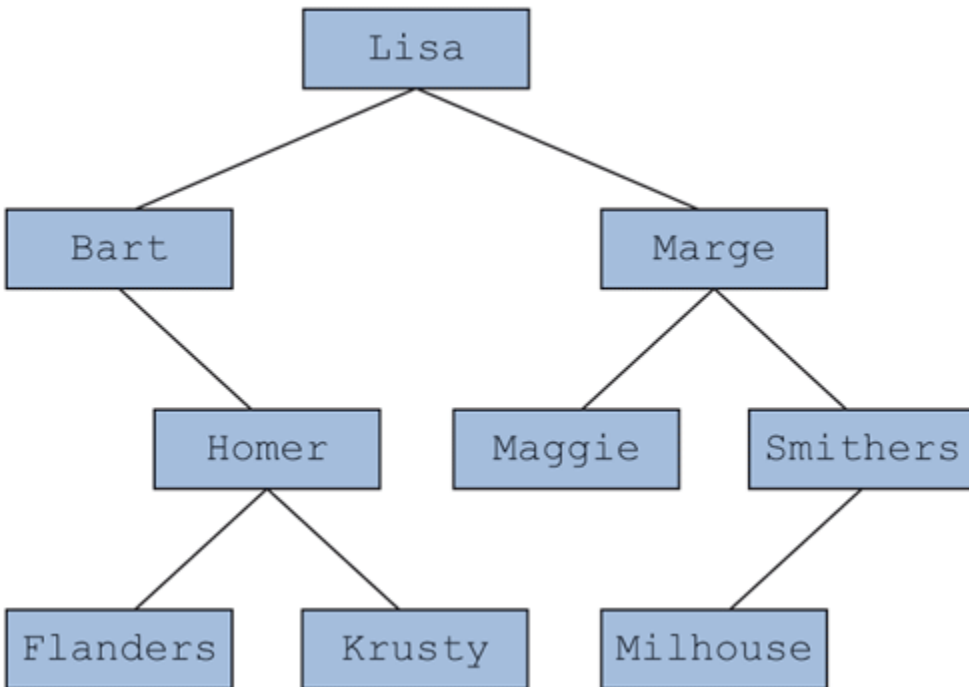
Property of a node which stores data n if all values stored in the left subtree are less than n and all values in the right subtree are greater than n . If duplicates are allowed, then all values in the left subtree must be less than or equal to n .

For a tree to qualify as a binary search tree, each node of the tree must have this property. This is not just a property of the overall root.

Binary Search Tree

A binary tree in which every node has the binary search tree property.

Here is a binary search tree of names:

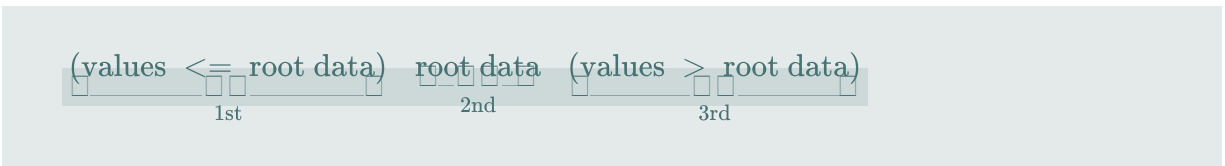


The names appear in the tree using alphabetical ordering (also known as *lexicographic ordering*). For example, the overall root stores `"Lisa"`, which means that all of the names in the left subtree are alphabetically less than or equal to `"Lisa"` and all names in the right subtree are alphabetically greater than `"Lisa"`.

Consider the result you obtain when you perform an inorder traversal of the tree:

```
Bart, Flanders, Homer, Krusty, Lisa, Maggie, Marge, Milhouse,  
Smithers
```

This list of names is in alphabetical order. This result isn't terribly surprising when you think about the binary search tree property. In a binary search tree, you know that values which are alphabetically less than or equal to the root appear in the left subtree and that values which are alphabetically greater appear in the right subtree. For any given subtree, an inorder traversal traverses the left subtree first, then the root, then the right subtree, which means you visit the nodes in this order:



Therefore, the root value is printed at just the right point in time (after the values that are alphabetically less than or equal to it and before the values that are alphabetically greater than it). And because every node in the tree has the binary search tree property, you know that the value stored in each node is printed in the correct position relative to the values that appear below it in the tree. The overall result is that each data value is printed at just the right point in time relative to the rest of the data.

This observation highlights one of the useful properties of a binary search tree. Once it is constructed, we can traverse the values in sorted order by performing an inorder traversal of the tree.

Building a Binary Search Tree

We can borrow many elements of the `IntTree` class that we wrote in the previous section to produce a new `IntSearchTree` class. The primary traversal in which we will be interested is the inorder traversal, so we can include it in our new class and rename it with the simpler name `print`.

For this new class, it is best to use the concept of a data invariant, as introduced in [Chapter 8](#). We can guarantee that the binary search tree property is true for every node in our tree by making sure that we never violate this invariant relationship. That will simplify our code because we won't have to test to make sure that our tree is a binary search tree.

We imagine that a client program will involve code like the following:

```
1  // This program tests the IntSearchTree class by
   constructing a
2  // binary search tree of integers and printing its contents
   as
3  // well as its structure.
4
5  import java.util.*;
6
7  public class IntSearchTreeClient {
8      public static void main(String[] args) {
9          Scanner console = new Scanner(System.in);
10         IntSearchTree numbers = new IntSearchTree();
```

```

11      System.out.print("Next int (0 to quit)? ");
12      int number = console.nextInt();
13      while (number != 0) {
14          numbers.add(number);
15          System.out.print("Next int (0 to quit)? ");
16          number = console.nextInt();
17      }
18      System.out.println();
19
20      System.out.println("Tree Structure:");
21      numbers.printSideways();
22      System.out.println("Sorted list:");
23      numbers.print();
24  }
25  }

```

This program prompts for a series of integers that are added one at a time into the tree until the user enters the sentinel value `0`. The program then prints the structure of the tree and the contents of the tree. The following is a sample execution. Notice that the numbers are printed in increasing order:

```

Next int (0 to quit)? 42
Next int (0 to quit)? 9
Next int (0 to quit)? 18
Next int (0 to quit)? 55

```

```
Next int (0 to quit)? 7
Next int (0 to quit)? 108
Next int (0 to quit)? 4
Next int (0 to quit)? 70
Next int (0 to quit)? 203
Next int (0 to quit)? 15
Next int (0 to quit)? 0
```

Tree Structure:

Category	Percentage
Current government	203
Previous government	108
Other	70
Other	55
Other	42
Other	18
Other	15
Other	9
Other	7
Other	4

Sorted list:

4 7 9 15 18 42 55 70 108 203

To begin with, you will need a constructor that constructs an empty tree. Remember that the empty tree is represented by the value `null`. In fact you could simply not have a constructor and Java would provide you with a zero-argument constructor that initializes the field `overallRoot` to `null`. But it's not a bad idea to include the constructor anyway for clarity:

```
public IntSearchTree() {  
    overallRoot = null;  
}
```

Then you have to write the `add` method. Because of our data invariant, you can assume that, each time the method is called, the existing tree is a binary search tree. You have to make sure that you add the value in an appropriate place so as to preserve the binary search tree property.

The `add` method will have the usual structure of a public method that the client calls with no mention of tree nodes and a private recursive method that takes a node as a parameter and that does the actual work. Thus, your pair of methods will look like this:

```
public void add(int value) {  
    add(overallRoot, value);  
}  
  
private void add(IntTreeNode root, int value) {  
    ...  
}
```

Recall that a binary tree is either empty or a root node with left and right subtrees. If it is empty, then you want to insert the value here. For example, initially the overall tree is empty and you insert the first

value at the top of the tree (replacing the `null` value with a reference to a new leaf node that contains the given value). So the private `add` method would start like this:

```
private void add(IntTreeNode root, int value) {  
    if (root == null) {  
        root = new IntTreeNode(value);  
    }  
    ...  
}
```

But what if it's not an empty tree? Then it must have a root node with some data. Let's assume that duplicates are allowed, so that you want to insert this value no matter what. In the case in which `root` is not `null`, you know that the new value has to be added to one of the subtrees. Which one? You can compare the value you are inserting to the data in the root to figure that out. If the value is less than or equal to the root's data, then you insert the value into the left subtree; otherwise, you insert into the right subtree. Your method will look something like this:

```
private void add(IntTreeNode root, int value) {  
    if (root == null) {  
        root = new IntTreeNode(value);  
    } else if (value <= root.data) {  
        // add to left
```

```
    } else {  
        // add to right  
    }  
}
```

This is the general structure that you want, but exactly how do you add to the left or right subtree? Some novices try to test things about the left or right subtree, as in the following example:

```
// overly complex version  
private void add(IntTreeNode root, int value) {  
    if (root == null) {  
        root = new IntTreeNode(value);  
    } else if (value <= root.data) {  
        if (root.left == null) {  
            root.left = new IntTreeNode(value);  
        } else {  
            // even more stuff  
        }  
    } else {  
        // add to right  
    }  
}
```

This is not a good way to approach the problem. If you're thinking recursively, you'll realize that it's another insertion task into either the

left or the right subtree. You can call the `add` method itself:

```
// simpler, but does not quite work
private void add(IntTreeNode root, int value) {
    if (root == null) {
        root = new IntTreeNode(value);
    } else if (value <= root.data) {
        add(root.left, value);
    } else {
        add(root.right, value);
    }
}
```

The logic of this code is almost correct. Unfortunately, in this form the tree always remains empty. The `add` method never inserts a value. The problem has to do with the parameter called `root`. The parameter `root` will store a copy of whatever value is passed into it. As a result, when you reassign `root`, it has no effect on the value that is passed into it.

There is a particular approach to solving this problem that is very helpful in writing binary tree code. Let's explore this issue and the technique in some detail before we try to fix this code.

The Pattern `x = change(x)`



Consider the following short program:

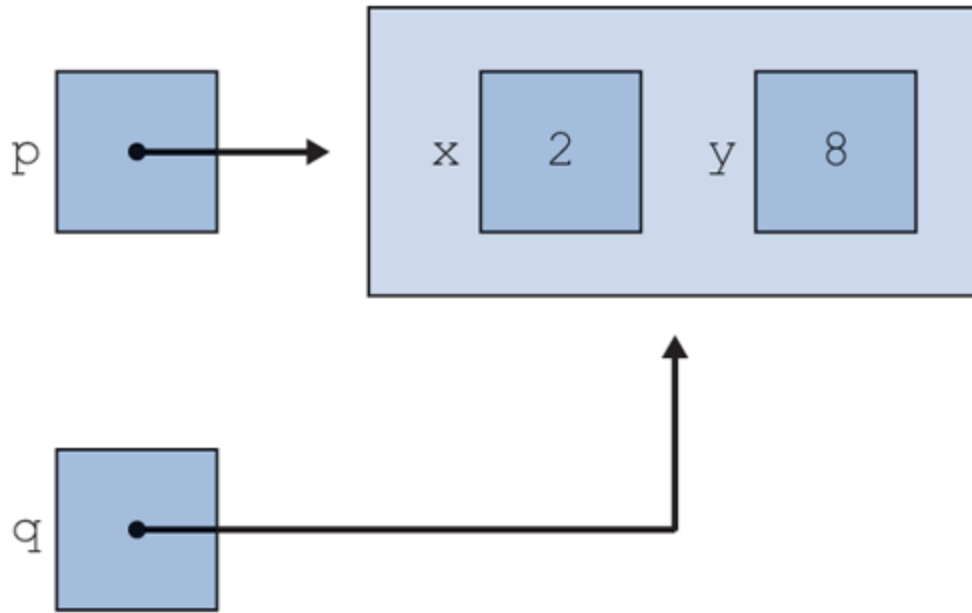
```
1  import java.awt.*;
2
3  public class PointTest {
4      public static void main(String[] args) {
5          Point p = new Point(2, 8);
6          System.out.println("p = " + p);
7          change(p);
8          System.out.println("now p = " + p);
9      }
10
11     public static void change(Point q) {
12         q.translate(3, 5);
13         q = new Point(-7, -14);
14         System.out.println("q = " + q);
15     }
16 }
```

The code in `main` constructs a `Point` object and passes it as a parameter. Inside the method we translate the coordinates of the `Point` object. This change is reflected in the original `Point` object because when we pass an object as a parameter, the parameter gets a copy of the reference to the object.

But what about the final line of the method that constructs a new `Point`? Does that change the `main` method's variable `p`? The answer is no. The overall output for this version of the program is as follows:

```
p = java.awt.Point[x=2,y=8]
q = java.awt.Point[x=-7,y=-14]
now p = java.awt.Point[x=5,y=13]
```

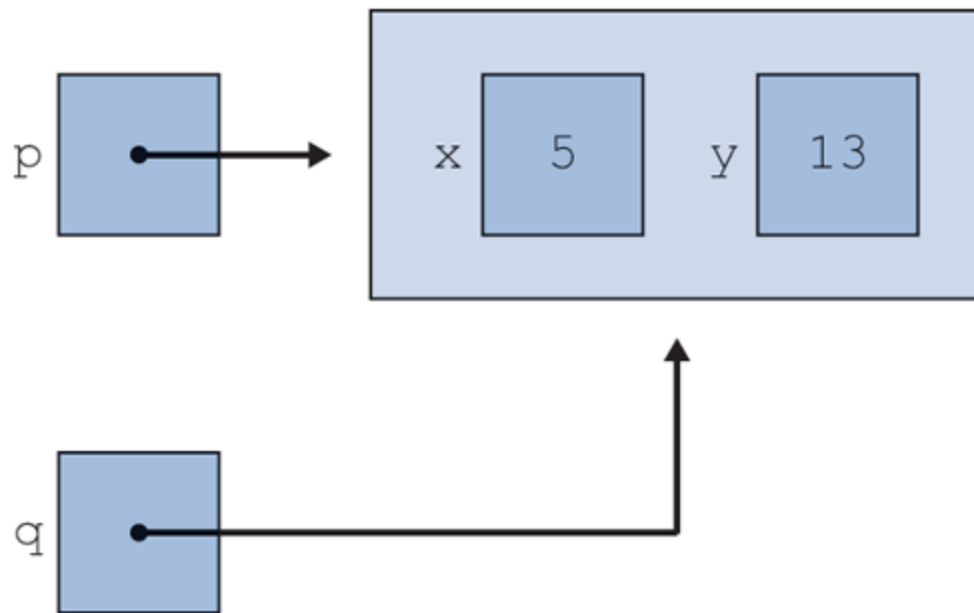
One of the changes (translating the coordinates) has an effect, but the other (constructing a new `Point`) does not. This happens because when we pass an object as a parameter, the parameter gets a copy of the reference to the object. When we call the `change` method, the variable `q` is set up as a copy of the variable `p`:



This is a situation we have seen before. There are two references to the same object. We discussed this scenario in detail in the section on reference semantics in [Chapter 7](#). There is only one `Point` object here and we have two different ways of referring to it: using the variable `p` or using the variable `q`. That's why the method is able to change the object to which `p` refers when it executes this line of code:

```
q.translate(3, 5);
```

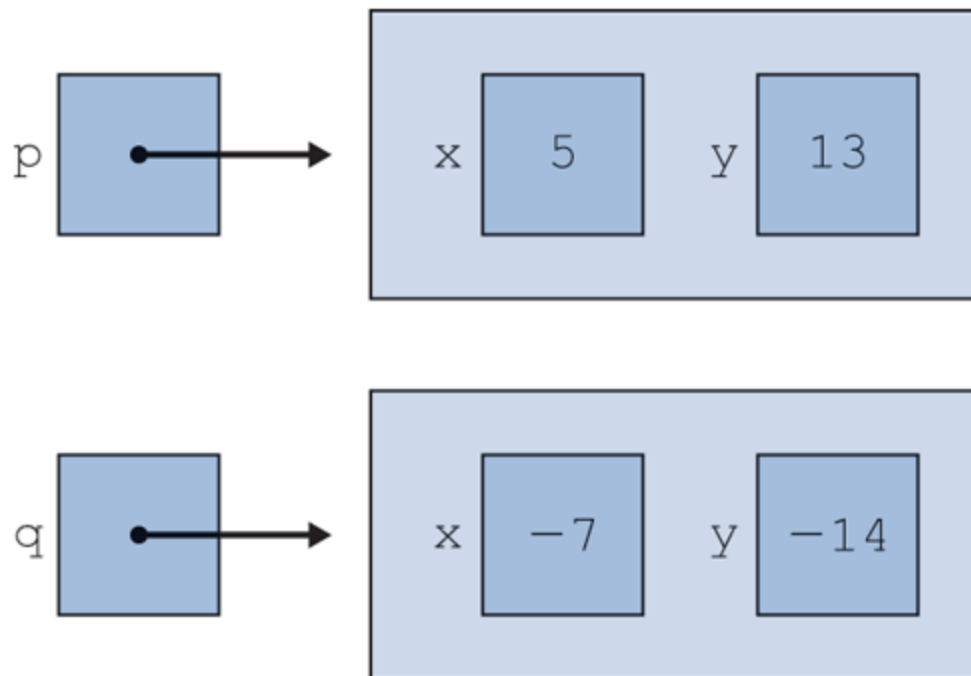
There are two different variables referring to the same object, so either variable is capable of changing the object:



But that doesn't mean that `q` has the power to change `p` itself. It can change the object to which `p` refers (the `Point`), but it can't change the value that is stored in `p` (the reference, or pointer; or the arrow, or cell phone number). When we change `q` with the following line of code, it has no effect on `p`:

```
q = new Point(-7, -14);
```

This code gives a new value to `q`, but not to `p`, as you can see in the following diagram:



There are many ways to try to fix this, but there is a particular approach called the `x = change(x)` idiom that solves the problem rather nicely. We change the method so that it returns the value of the parameter just before it exits. This modification also requires us to change the return type for the method:

```
public static Point change(Point q) {  
    q.translate(3, 5);  
    q = new Point(-7, -14);  
    System.out.println("q = " + q);  
    return q;  
}
```

Then we change the call on the method to the `x = change(x)` form:

```
p = change(p);
```

With these changes, the program produces the following output:

```
p = java.awt.Point[x=2,y=8]
q = java.awt.Point[x=-7,y=-14]
now p = java.awt.Point[x=-7,y=-14]
```

The change that occurs in the method propagates back to the `main` method.

Now let's return to the binary tree `add` method to see how we could apply this technique there. Currently the `add` method has a return type of `void`:

```
private void add(IntTreeNode root, int value) {
    ...
}
```

We can change it so that the last thing we do in the method is to return the value of `root`, which means we have to change the return type to `IntTreeNode`:

```
private IntTreeNode add(IntTreeNode root, int value) {  
    ...  
    return root;  
}
```

Then we change every call on `add` to match the `x = change(x)` form.
Recall our old public method:

```
public void add(int value) {  
    add(overallRoot, value);  
}
```

The code now becomes:

```
public void add(int value) {  
    overallRoot = add(overallRoot, value);  
}
```

The idea is that we pass the value of `overallRoot` to the `add` method and it passes back the value of the parameter, which might be the old value or might be a new value. We reassign `overallRoot` to this value passed back by `add`. That way, if the method changes the value of the parameter, `overallRoot` gets updated to that new value. If it doesn't change the value of `overallRoot`, then we are simply

assigning a variable to the value that it already has (effectively saying `"x = x"`), which has no effect.

Two other calls on `add` inside the method itself need to be updated in a similar manner:

```
private IntTreeNode add(IntTreeNode root, int value) {  
    if (root == null) {  
        root = new IntTreeNode(value);  
    } else if (value <= root.data) {  
        root.left = add(root.left, value);  
    } else {  
        root.right = add(root.right, value);  
    }  
    return root;  
}
```

After we make these changes, the `add` method works properly.

This `x = change(x)` idiom is very powerful and simplifies a lot of binary tree code, so it is important to learn how to use it well.

Searching the Tree

Once we have built a binary search tree, we will want to do more than just printing its contents or structure. One of the most common operations we might want to perform is to search the tree to see whether it contains a particular value.

The method to perform this search should return a `boolean` result and you will once again want to have a public/private pair of methods, so the basic form of your solution will be as follows:

```
public boolean contains(int value) {  
    return contains(overallRoot, value);  
}  
  
private boolean contains(IntTreeNode root, int value) {  
    ...  
}
```

Now you just have to write the body of the private method. An empty tree again serves as the easy case. If you're asked whether the empty tree contains a value, the answer is no (`false`):

```
private boolean contains(IntTreeNode root, int value) {  
    if (root == null) {  
        return false;  
    }
```

```
    ...  
}
```

For the `add` method, you had two other cases. You either added to the left subtree or added to the right subtree. For this method, there are three cases, cases for searching the left and right subtree and the root:

```
private boolean contains(IntTreeNode root, int value) {  
    if (root == null) {  
        return false;  
    } else if (value == root.data) {  
        ...  
    } else if (value < root.data) {  
        ...  
    } else { // value > root.data  
        ...  
    }  
}
```

Now you just need to fill in the three different cases. If the value is less than `root.data`, then it is either in the left subtree or not in the tree at all. In other words, you want to search the left subtree to see whether it contains the value. You can accomplish this with a recursive call on `contains`. You can make a similar recursive call on `contains` to search the right subtree when the value is greater than

`root.data`. In the final case in which the value is equal to `root.data`, you should simply report that you found the value. In other words, the answer is `true`. Putting these pieces together, you end up with the following code:

```
private boolean contains(IntTreeNode root, int value) {  
    if (root == null) {  
        return false;  
    } else if (value == root.data) {  
        return true;  
    } else if (value < root.data) {  
        return contains(root.left, value);  
    } else { // value > root.data  
        return contains(root.right, value);  
    }  
}
```

A clever reader might notice that this method violates the principle of Boolean Zen that we introduced in [Chapter 5](#). This method can be written more concisely as a Boolean expression. For example, it can be written as a single expression as follows:

```
private boolean contains(IntTreeNode root, int value) {  
    return root != null && (root.data == value ||  
        (value < root.data && contains(root.left, value))  
    )  
}
```

```
        (value >= root.data && contains(root.left,
value)));
    }
```

In this case, it is debatable whether the single expression is more readable than the `if/else` version of the code. This is somewhat a matter of personal taste, so different programmers will prefer one version over the other, but our conclusion was that the longer version is actually easier to read in this case.

Here is the complete implementation of the `IntSearchTree` class:

```
1  // This class stores int values in a binary search tree.
2  // Duplicates are allowed. Each node of the tree has the
binary
3  // search tree property.
4
5  public class IntSearchTree {
6      private IntTreeNode overallRoot;
7
8      // post: constructs an empty tree
9      public IntSearchTree() {
10         overallRoot = null;
11     }
12
13     // post: value is added to overall tree so as to
preserve the
```

```
14      //      binary search tree property
15      public void add(int value) {
16          overallRoot = add(overallRoot, value);
17      }
18
19      // post: value is added to given tree so as to preserve
the
20      //      binary search tree property
21      private IntTreeNode add(IntTreeNode root, int value) {
22          if (root == null) {
23              root = new IntTreeNode(value);
24          } else if (value <= root.data) {
25              root.left = add(root.left, value);
26          } else {
27              root.right = add(root.right, value);
28          }
29          return root;
30      }
31
32      // post: returns true if overall tree contains value
33      public boolean contains(int value) {
34          return contains(overallRoot, value);
35      }
36
37      // post: returns true if given tree contains value
38      private boolean contains(IntTreeNode root, int value) {
39          if (root == null) {
40              return false;
```

```
41         } else if (value == root.data) {
42             return true;
43         } else if (value < root.data) {
44             return contains(root.left, value);
45         } else { // value > root.data
46             return contains(root.right, value);
47         }
48     }
49
50
51     // post: prints the tree contents using an inorder
traversal
52     public void print() {
53         printInorder(overallRoot);
54         System.out.println();
55     }
56
57     // post: prints contents of the tree with given root
using an
58         //         inorder traversal
59     private void printInorder(IntTreeNode root) {
60         if (root != null) {
61             printInorder(root.left);
62             System.out.print(root.data + " ");
63             printInorder(root.right);
64         }
65     }
66
```

```

67      // post: prints the tree contents, one per line,
following an
68      //      inorder traversal and using indentation to
indicate
69      //      node depth; prints right to left so that it
looks
70      //      correct when the output is rotated.
71      public void printSideways() {
72          printSideways(overallRoot, 0);
73      }
74
75      // post: prints in reversed preorder the tree with
given
76      //      root, indenting each line to the given level
77      private void printSideways(IntTreeNode root, int level)
{
78          if (root != null) {
79              printSideways(root.right, level + 1);
80              for (int i = 0; i < level; i++) {
81                  System.out.print("    ");
82              }
83              System.out.println(root.data);
84              printSideways(root.left, level + 1);
85          }
86      }
87  }

```

Binary Search Tree Complexity

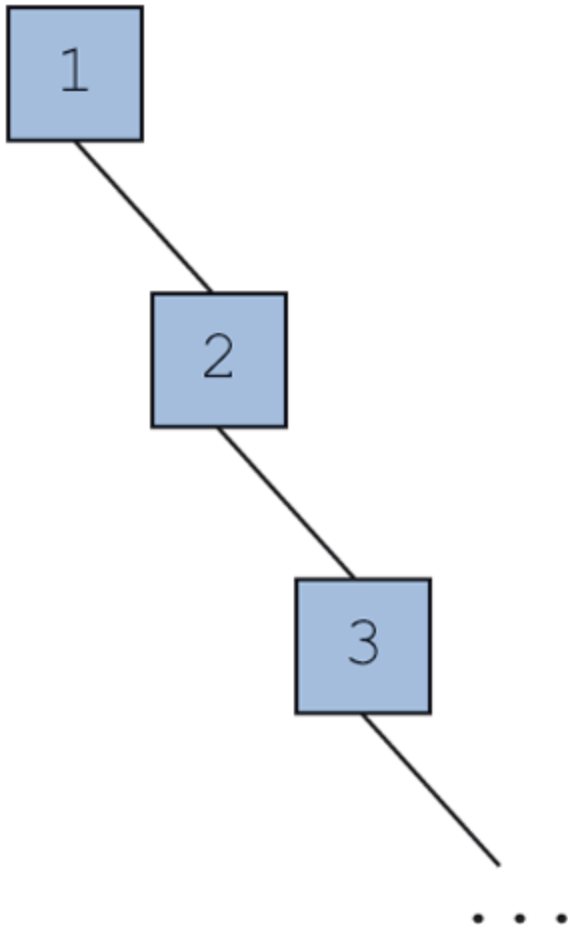
Each level of a binary search tree can hold twice as many nodes as the level above it. The one root node can have two children, four grandchildren, eight great-grandchildren, and so on. As a result, we can store a lot of values in a tree that has a relatively small number of levels.

The number of nodes and the number of levels have an important relationship. If you have k levels in a tree, then you can store approximately 2^k values in the tree. Going in the other direction, if you want to include N nodes in a tree, you will need approximately $\log N$ levels for the tree. We normally use this second way of thinking about the relationship.

It may not seem immediately obvious how powerful this is because you may not realize how much smaller $\log N$ is than N . Consider some examples. To store a thousand values, you would need around 10 levels. Increase the number of values to a million, and you need 20 levels. Increase the number to a billion, and you need 30 levels. To store a trillion values, you need only 40 levels for the tree. As a result, binary search trees tend to be relatively short in height, even when they store vast amounts of data.

It is also important to realize that the insertion algorithm we have developed in this chapter does not guarantee that the tree will store data efficiently in the minimum number of levels. While it's true that

you can store a trillion values with just 40 levels, you might end up with considerably more levels than that. Consider, for example, what happens if the data are inserted in sorted order. If you build a binary search tree by inserting the sequential numbers 1 through 1000, you end up with a tree that looks like this one:



We refer to this as a *degenerate tree*. It's technically still a tree, but it's not a very good tree because it makes no use of the two branching possibilities. This is really a linked list turned on its side. You can get similar degenerate trees if the values are inserted in decreasing order, which produces a tree that expands indefinitely in the left direction.

If the values are inserted into a binary search tree in a random order, then the odds of getting a degenerate tree are fairly low. If you take a more advanced course in data structures, you will study techniques for ensuring that the tree is balanced. For example, the `TreeMap` and `TreeSet` structures in the Java class libraries are implemented as what are called red/black trees to guarantee that the resulting binary search tree is balanced.

Let's set aside the issue of degenerate trees and think about the case for randomized data or for binary search trees that guarantee balance. One important property of the binary search tree is that each insertion into the tree requires you to descend from level to level until you find an open spot to insert the value. Thus, insertion into a balanced binary search tree is an $O(\log N)$ operation. Similarly, finding a value in a balanced binary search tree involves descending from level to level, so it also is an $O(\log N)$ operation. To search for a value in a normal binary tree, you would have to search the entire tree. Although we haven't shown the solution here, we'll mention that removing a value from a balanced binary search tree can also be accomplished as an $O(\log N)$ operation.

The binary search tree, then, offers insertion, search, and removal all in $O(\log N)$ time. That means that if you are working with a trillion values, for example, you can add a new value, search for a value, or remove a value, all within around 40 steps. That makes the binary search tree a very efficient structure to use for these operations.

17.5 `SearchTree<E>`

In the previous section, we developed code for a binary search tree of integers. You might want to build other kinds of binary search trees with different kinds of data and you wouldn't want to make many copies of essentially the same code (one for integers, one for strings, etc.). Instead, you want to write the code in a more generic way.

We want to be able to build a search tree for any class that implements the `Comparable` interface. We want a class that we could call `SearchTree<E>` (for some element type `E`). Here is a client program which constructs a `SearchTree<String>` that puts words into alphabetical order and a `SearchTree<Integer>` that puts numbers into numerical order.

```
1 // This program uses the SearchTree class to construct a
binary
2 // search tree of strings and a binary search tree of
integers
3 // and print out each.
4
5 import java.util.*;
6
7 public class SearchTreeClient {
```

```
8      public static void main(String[] args) {
9          Scanner console = new Scanner(System.in);
10         SearchTree<String> names = new SearchTree<>();
11         System.out.print("Name (blank to quit)? ");
12         String name = console.nextLine();
13         while (name.length() > 0) {
14             names.add(name);
15             System.out.print("Name (blank to quit)? ");
16             name = console.nextLine();
17         }
18         System.out.println();
19         System.out.println("Alphabetized list:");
20         names.print();
21         System.out.println();
22
23         SearchTree<Integer> numbers = new SearchTree<>();
24         System.out.print("Next int (0 to quit)? ");
25         int number = console.nextInt();
26         while (number != 0) {
27             numbers.add(number);
28             System.out.print("Next int (0 to quit)? ");
29             number = console.nextInt();
30         }
31         System.out.println();
32         System.out.println("Sorted list:");
33         numbers.print();
34     }
35 }
```

Here is a sample log of execution:

```
Name (blank to quit)? Leonard
Name (blank to quit)? Sheldon
Name (blank to quit)? Howard
Name (blank to quit)? Penny
Name (blank to quit)? Raj
Name (blank to quit)? Leslie
Name (blank to quit)? Ma
Name (blank to quit)?
```

```
Alphabetized list:
```

```
Howard
Leonard
Leslie
Ma
Penny
Raj
Sheldon
```

```
Next int (0 to quit)? 38
Next int (0 to quit)? 19
Next int (0 to quit)? -47
Next int (0 to quit)? 2
Next int (0 to quit)? 42
Next int (0 to quit)? 13
Next int (0 to quit)? 9
```

```
Next int (0 to quit)? 0
```

```
Sorted list:
```

```
-47
```

```
2
```

```
9
```

```
13
```

```
19
```

```
38
```

```
42
```

To generate the `SearchTree` class, we can start with the `IntSearchTree` class and convert it into generic form.

First we need a node class for the tree. Our node class is almost the same as the `IntTreeNode` class, but instead of having data of type `int`, we have data of type `E`, and instead of saying `IntTreeNode`, we have to say `SearchTreeNode<E>` in most places other than the constructor headers:

```
1 public class SearchTreeNode<E> {  
2     public E data;           // data stored in this  
node  
3     public SearchTreeNode<E> left; // left subtree  
4     public SearchTreeNode<E> right; // right subtree  
5 }
```

```

6      // post: constructs a leaf node with given data
7      public SearchTreeNode(E data) {
8          this(data, null, null);
9      }
10
11     // post: constructs a node with the given data and
links
12     public SearchTreeNode(E data, SearchTreeNode<E> left,
13                           SearchTreeNode<E> right) {
14         this.data = data;
15         this.left = left;
16         this.right = right;
17     }
18 }

```

So what about the `SearchTree` class? You can get a pretty close approximation of the code for this class by replacing all occurrences of `int` with `E` and replacing all occurrences of `IntTreeNode` with `SearchTreeNode<E>`. But you have to make a few adjustments. First, the private `add` method looks like this after the substitution:

```

private SearchTreeNode<E> add(SearchTreeNode<E> root, E value)
{
    if (root == null) {
        root = new SearchTreeNode<>(value);
    } else if (value <= root.data) {
        root.left = add(root.left, value);
    }
}

```

```
    } else {  
        root.right = add(root.right, value);  
    }  
    return root;  
}
```

The problem here is that you have to call the `compareTo` method rather than using the `<=` operator. You'll replace the line of code:

```
} else if (value <= root.data) {
```

with:

```
} else if (value.compareTo(root.data) <= 0) {
```

You have to make a similar change in the private `contains` method. You have to use calls on the `compareTo` method rather than simple comparisons. Because the method has more than one comparison, in this case it makes sense to call `compareTo` once and store it in a local variable:

```
private boolean contains(SearchTreeNode<E> root, E value) {  
    if (root == null) {  
        return false;  
    }
```

```

    } else {
        int compare = value.compareTo(root.data);
        if (compare == 0) {
            return true;
        } else if (compare < 0) {
            return contains(root.left, value);
        } else {
            return contains(root.right, value);
        }
    }
}
}
}

```

You have to make one final change. Java knows only that these data values are of some generic type `E`. As a result, the calls on `compareTo` in your `add` and `contains` methods generate compiler errors. You could fix this problem with a cast. For example, you could replace the line:

```
int compare = value.compareTo(root.data);
```

with:

```
int compare = ((Comparable<E>) value).compareTo(root.data);
```

A better approach is to modify the class header to include this information. You want to add the constraint that the class `E` implements the `Comparable` interface. You specify that constraint by modifying the header as follows:

```
public class SearchTree<E extends Comparable<E>> {  
    ...  
}
```

It's odd that Java has you use the keyword `extends` because you want it to implement the interface, but that's how generics work in Java. If you are defining a class, you make a distinction between the cases in which the new class extends another class and those in which it implements an interface. But in generic declarations, you use the word `extends` for both kinds of extension.

This is a fairly quick explanation of a complex topic. We don't have time to explain the details of programming with generics in detail in this book, although there are many excellent online tutorials that you can find by entering "Java generics" into your favorite web browser.

The complete code for the `SearchTree` class can be found at <http://www.buildingjavaprograms.com>.

Chapter Summary

A binary tree is a linked structure in which each node refers to two other nodes. A tree is either empty (`null`) or a node with references to two other trees, called its subtrees.

The root is the top node of a tree. A leaf node is a node that has no children. A branch node is a node that has at least one child. Each node has a level related to the number of links that are between it and the root of the tree.

A tree node object stores a data value and left/right references to other nodes. An overall tree object stores a reference to a single tree node as its overall root.

A traversal is an examination of all elements in a binary tree. Traversals are commonly done in three orders: preorder, inorder, and postorder. The difference between the orders relates to when a given node is examined relative to its left/right subtrees.

Tree methods are often recursive and are often implemented with the use of a public/private pair, in which the private method accepts

a reference to a tree node as a parameter. This technique allows the method to recursively operate on a given portion of the overall tree.

The elements of a binary search tree are arranged in order, with smaller elements on the left and larger elements on the right. Search trees are useful for implementing collections that can be searched quickly, such as sets and maps.

To add a node to a binary tree, traverse the tree to the left if the new element is smaller than the current node or to the right if it is larger. The end of this traversal will reveal the proper place to add the new node.

To search for a value in a binary tree, traverse the tree going left if the target value is smaller than the current node or right if it is larger. Because of the way in which the tree is ordered, the end of this traversal will either find the target's node or find a `null` dead end, in which case the value is not in the tree.

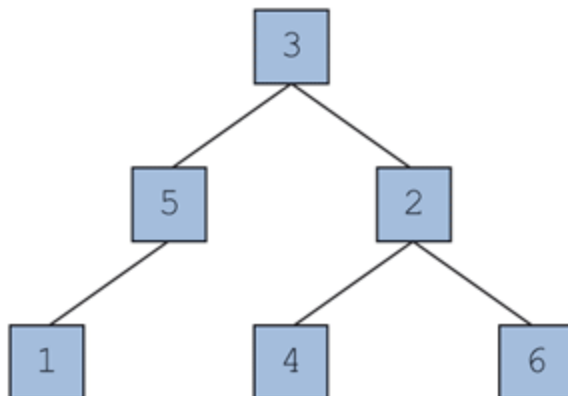
The `x = change(x)` is a pattern in which a recursive method is passed an object or value in its initial state (called `x` here) and returns the new state of `x`. This technique is often used in recursive tree methods; the recursive call accepts a node parameter and later returns the potentially changed node.

A tree can be made to store elements of any object type `E` rather than just integers. To do this, the tree class must be made into a generic class, and the generic type parameter `E` must be constrained so that it is a class which implements the `Comparable<E>` interface.

Self-Check Problems

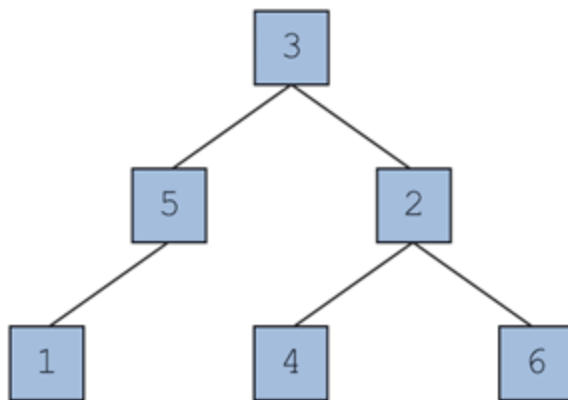
Section 17.1: Binary Tree Basics

1. How many roots can a tree have? Does a tree have more branches or leaves?
2. Draw a tree that has twice as many leaves as branches and one that has the same number of leaves as branches.
3.
 - a. How many levels does the given tree have?
 - b. How many branches does it have, and which nodes are they?
 - c. How many leaves does it have, and which nodes are they?
 - d. Which node is the root of the tree?
 - e. Which node(s) are the sibling(s) of the node storing the value 2? Which nodes are its children?

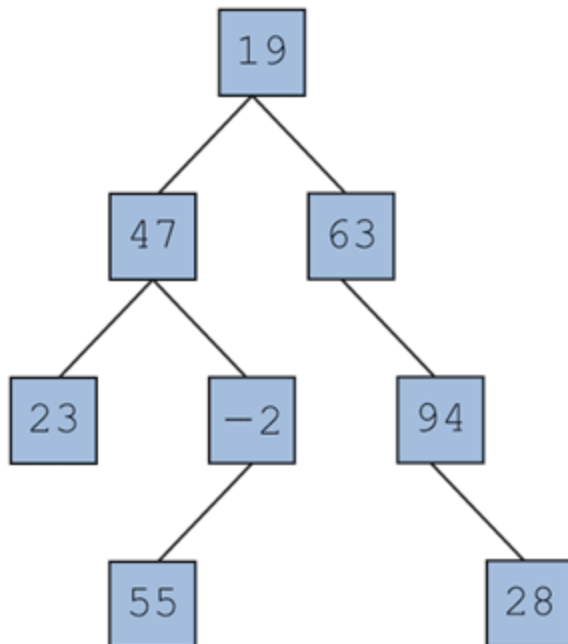


Section 17.2: Tree Traversals

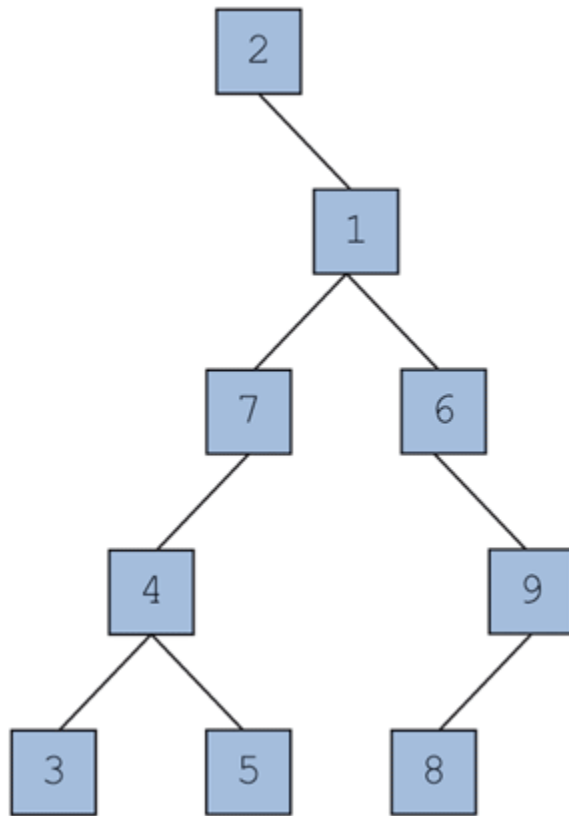
For each of the next three problems, write the elements of the given tree in the order in which they would be seen by a preorder, inorder, and postorder traversal.



4.



5.



6.

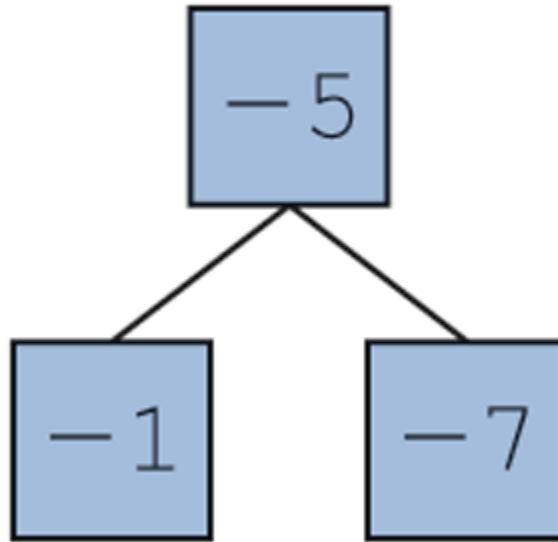
7. What would happen if we removed the `root != null` test from the `printPreorder` method?
8. Write a method called `printPostorder` that could be added to the `IntTree` class and that prints a postorder traversal of the tree.
9. Write a method called `printMirror` that could be added to the `IntTree` class and that prints a backward inorder traversal of the tree. That is, for a given node, it examines the right subtree, then the node itself, then the left subtree.

Section 17.3: Common Tree Operations

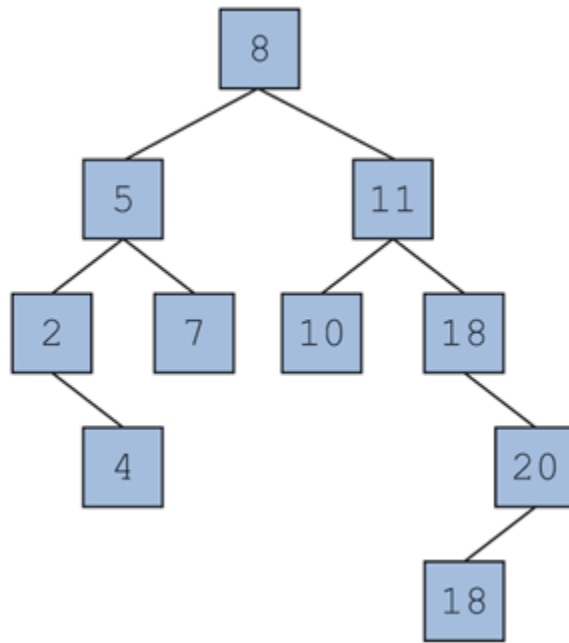
10. Why do many recursive tree methods use a public/private pair? What is the difference between the header of the public method and that of the private method?
11. Write a method called `size` that could be added to the `IntTree` class and that returns the total number of nodes in the tree.
12. Write methods called `min` and `max` that could be added to the `IntTree` class and that return the smallest and largest values in the tree, respectively. For example, if a variable called `tree` stores the values shown in [Self-Check Problem 5](#), the call of `tree.min()` should return `-2` and the call of `tree.max()` should return `94`. If the tree is empty, the methods should throw an `IllegalStateException`.
13. Write a method called `countBranches` that could be added to the `IntTree` class and that returns the total number of branch nodes in the tree. A branch node is any node that is not a leaf node. (Hint: Look at the code for `countLeaves` written in [Section 17.3](#).)

Section 17.4: Binary Search Trees

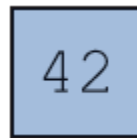
14. What is the difference between a regular binary tree and a binary search tree?
15. Which of the following trees are valid binary search trees?



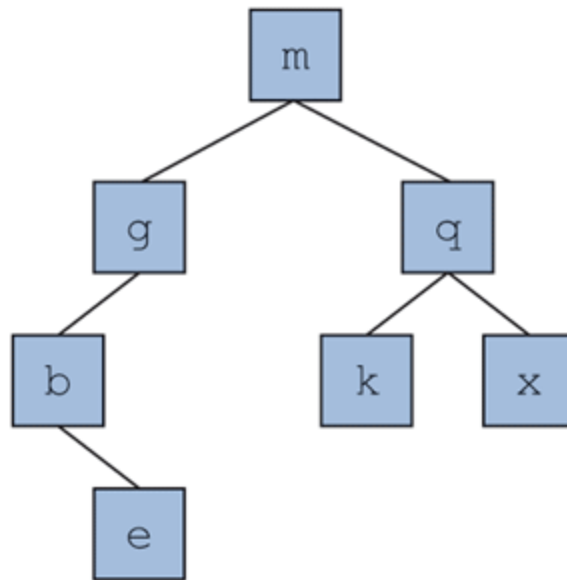
a.



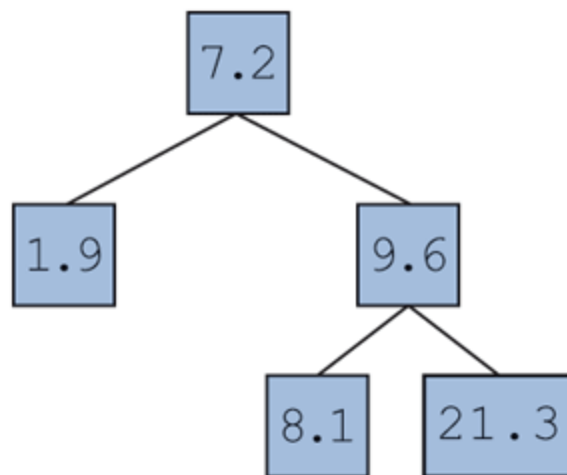
b.



c.



d.



e.

16. What will be true about the results of an inorder traversal of a binary search tree?

For each of the next four problems, draw the binary search tree that would result if the given elements were added to an empty binary search tree in the given order. Then write the

elements of the tree in the order that they would be visited by each kind of traversal (preorder, inorder, and postorder).

17. Leia, Boba, Darth, R2D2, Han, Luke, Chewy, Jabba
18. Meg, Stewie, Peter, Joe, Lois, Brian, Quagmire, Cleveland
19. Kirk, Spock, Scotty, McCoy, Chekov, Uhuru, Sulu, Khaaaaan!
20. Lisa, Bart, Marge, Homer, Maggie, Flanders, Smithers, Milhouse
21. Why does the `add` method of a binary search tree need to return the newly added/created node?
22. What is the `x = change(x)` pattern, and how is it used with binary trees?
23. How many nodes at most would be examined in a call to `contains` on a perfect binary search tree of height N ?
24. Consider the following implementation of the `contains` method. How does it differ from the one we showed in **Section 17.4**? Is it better or worse, and why?

```
private boolean contains(IntTreeNode root, int value) {  
    if (root == null) {  
        return false;  
    } else if (value == root.data) {  
        return true;  
    } else {  
        return contains(root.left, value) ||  
            contains(root.right, value);  
    }  
}
```

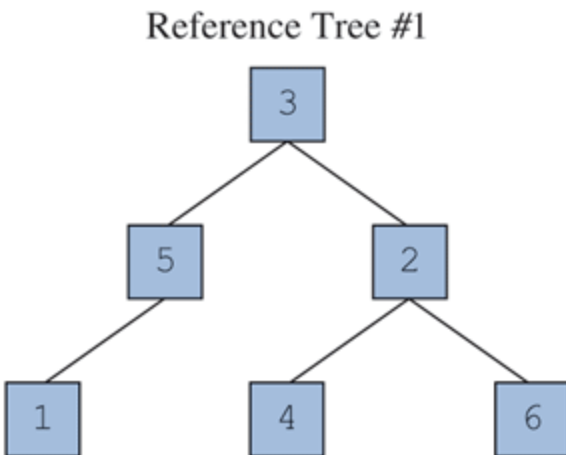
25. Rewrite the `min` and `max` methods from **Self-Check Problem 12**  so that they will work on a binary search tree. The methods should take advantage of the fact that the tree is sorted and should not examine nodes unless necessary.

Section 17.5: `SearchTree<E>`

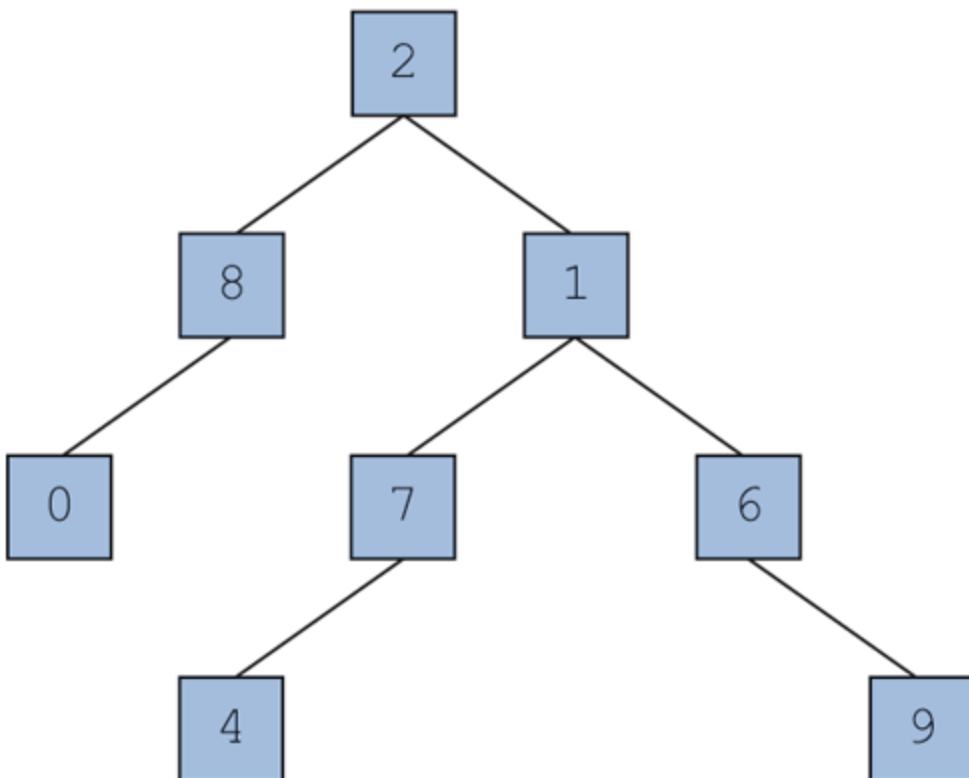
26. What are some changes that need to be made to the tree class to convert it from storing integers to storing objects of type `E`?
27. What kind of changes would you make to add an iterator to our binary tree class?

Exercises

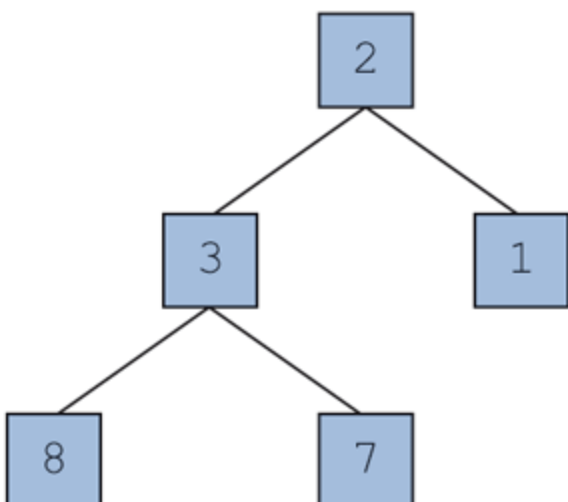
Each of the following exercises is a method to be added to the `IntTree` class from this chapter. You may define additional private methods to implement your public method if necessary. Several problem descriptions refer to the following reference binary trees:



Reference Tree #2



Reference Tree #3



1. Write a method called `countLeftNodes` that returns the number of left children in the tree. A left child is a node that appears as the root of the left-hand subtree of another node. For example, reference tree #1 has 3 left children (the nodes storing the values 5, 1, and 4).
2. Write a method called `countEmpty` that returns the number of empty branches in a tree. An empty tree is considered to have one empty branch (the tree itself). For nonempty trees, your methods should count the total number of empty branches among the nodes of the tree. A leaf node has two empty branches, a node with one nonempty child has one empty branch, and a node with two nonempty children has no empty branches. For example, reference tree #1 has 7 empty branches (two under the value 1, one under 5, and two under each of 4 and 6).
3. Write a method called `depthSum` that returns the sum of the values stored in a binary tree of integers weighted by the depth of each value. The method should return the value at the root, plus 2 times the values stored at the next level of the tree, plus 3 times the values stored at the next level of the tree, and so on. For example, the depth sum of reference tree #1 would be computed as $(1 * 3) + (2 * (5 + 2)) + (3 * (1 + 4 + 6)) = 50$.
4. Write a method called `countEvenBranches` that returns the number of branch nodes in a binary tree that contain even numbers. A branch node has one or two children (i.e., it is not a leaf). For example, if a variable `t` refers to reference tree

#2, then the call `t.countEvenBranches()` should return `3` because there are three branch nodes with even values (`2`, `8`, and `6`). Notice that leaf nodes with even values are not included (the nodes storing `0` and `4`).

5. Write a method called `printLevel` that accepts an integer parameter `n` and prints the values at level `n` from left to right, one per line. We will use the convention that the overall root is at level `1`, its children are at level `2`, and so on. If there are no values at the level, your method should produce no output. Your method should throw an `IllegalArgumentException` if it is passed a value for a level that is less than 1. For example, if a variable `t` refers to reference tree #2, then the call of `t.printLevel(3);` would produce the following output:

```
0
7
6
```

6. Write a method called `printLeaves` that prints to `System.out` the leaves of a binary tree from right to left. More specifically, the leaves should be printed in the *reverse* order that they would be printed using any of the standard traversals. If the tree is empty, your method should produce the output `"no leaves"`. For example, if a variable `t` refers to reference tree #2, the call of `t.printLeaves();` should produce the following output:

```
leaves: 9 4 0
```

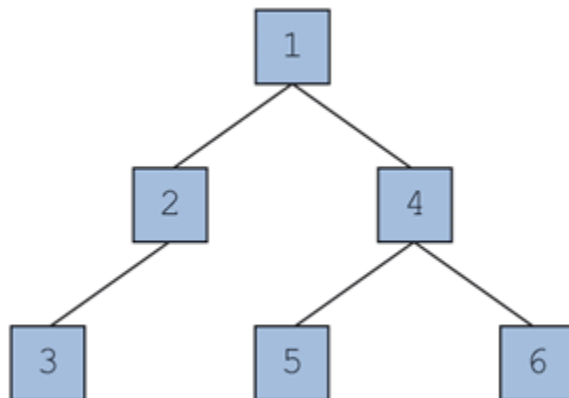
7. Write a method called `isFull` that returns `true` if a binary tree is full and `false` if it is not. A full binary tree is one in which every node has 0 or 2 children. For example, reference trees #1 and #2 are not full, but #3 is full. By definition, the empty tree is considered full.
8. Write a `toString` method for a binary tree of integers. The method should return `"empty"` for an empty tree. For a leaf node, it should return the data in the node as a string. For a branch node, it should return a parenthesized `String` that has three elements separated by commas: the data at the root, a string representation of the left subtree, and then a string representation of the right subtree. For example, if a variable `t` refers to reference tree #2, then the call `t.toString()` should return the following `String` (without the surrounding quotes):

```
"(2, (8, 0, empty), (1, (7, 4, empty), (6, empty, 9)))"
```

9. Write a method called `equals` that accepts another binary tree of integers as a parameter and compares the two trees to see whether they are equal to each other. For example, if variables of type `IntTree` called `t1` and `t2` have been initialized, then `t1.equals(t2)` will return `true` if the trees are

equal and `false` otherwise. Two empty trees are considered to be equal to each other.

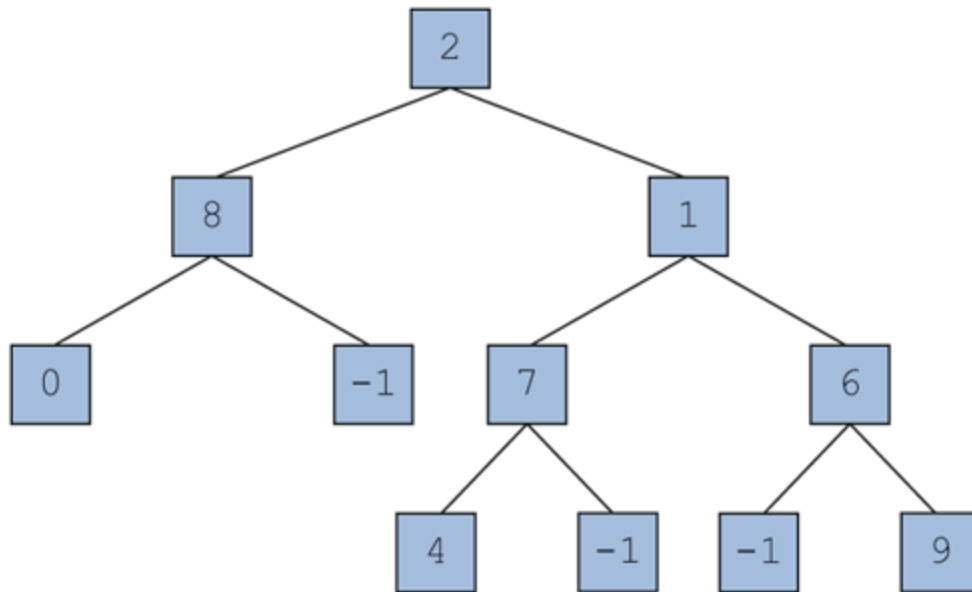
10. Write a method called `doublePositives` that doubles all data values greater than 0 in a binary tree of integers.
11. Write a method called `numberNodes` that changes the data stored in a binary tree, assigning sequential integers starting with `1` to each node so that a preorder traversal will produce the numbers in order (`1, 2, 3,` etc.). For example, if a variable `t` refers to reference tree #1, the call of `t.numberNodes();` would overwrite the existing data, assigning values from `1` to `6` to the nodes so that a preorder traversal of the tree would produce `1, 2, 3, 4, 5, 6` as shown in the following diagram. Do not change the structure of the tree, only the values stored in the data fields. Your method should return the number of nodes in the tree.



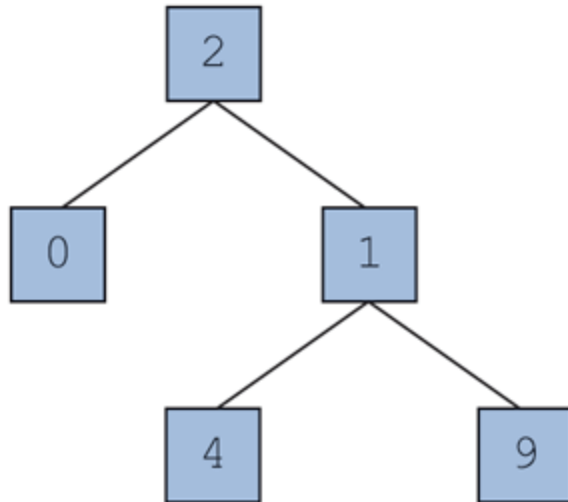
12. Write a method called `removeLeaves` that removes the leaves from a tree. A leaf is a node that has empty left and right subtrees. If your method is called on an empty tree, the

method does not change the tree because there are no nodes of any kind (leaf or not).

13. Write a method called `copy` that returns a reference to a new `IntTree` that is an independent copy of the original tree. Do not change the original tree.
14. Write a method called `completeToLevel` that accepts an integer n as a parameter and that adds nodes to a tree to complete the first n levels. A level is complete if every possible node at that level is not `null`. We will use the convention that the overall root is at level `1`, its children are at level `2`, and so on. You should preserve any existing nodes in the tree. Any new nodes added to the tree should contain the value `-1`. Your method should throw an `IllegalArgumentException` if it is passed a value for a level that is less than `1`.
For example, if a variable called `t` refers to reference tree #2 and you make the call of `t.completeToLevel(4);`, the tree should change to the following tree:

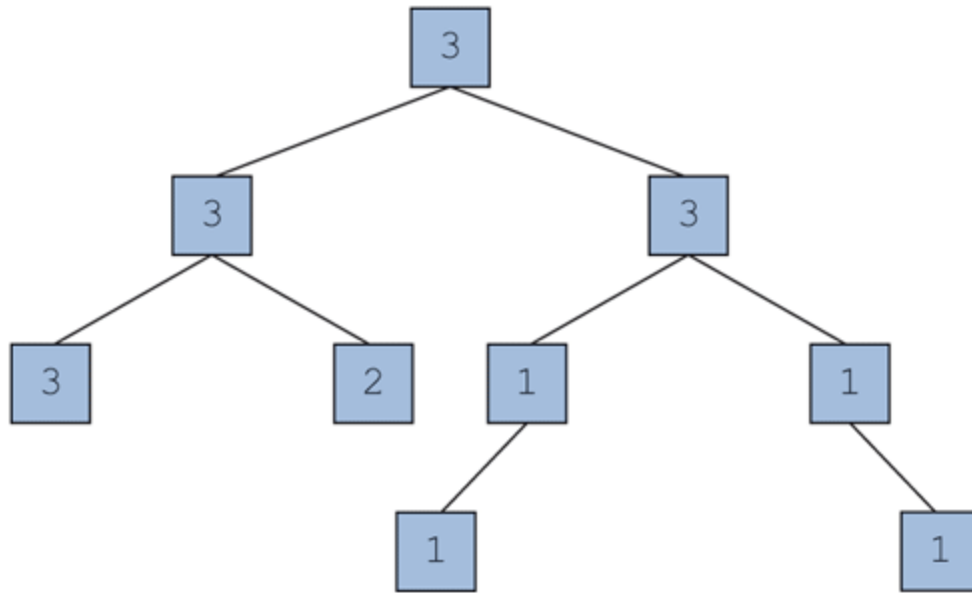


15. Write a method called `trim` that accepts minimum and maximum integers as parameters and removes from the tree any elements that are not within that range inclusive. For this method you should assume that your tree is a binary search tree and that its elements are in valid binary search tree order. Your method should maintain the binary search tree ordering property of the tree. This property is important for solving this problem.
16. Write a method called `tighten` that eliminates branch nodes that have only one child. Each such node should be replaced by its only child. (This can lead to multiple replacements because the child might itself be replaced.) For example, if a variable called `t` refers to reference tree #2, the call of `t.tighten();` should leave `t` storing the following tree.

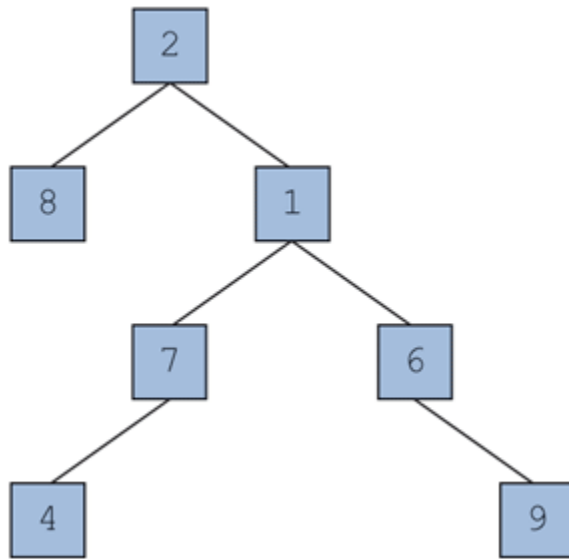


17. Write a method called `combineWith` that accepts another binary tree of integers as a parameter and combines the two trees into a new third tree that is returned. The new tree's structure should be a union of the structures of the two original trees; it should have a node in any location where there was a node in either of the original trees (or both). The nodes of the new tree should store an integer indicating which of the original trees had a node at that position (`1` if just the first tree had the node, `2` if just the second tree had the node, and `3` if both trees had the node). Your method should not change the structure or contents of either of the two original trees that are being combined.
- For example, suppose `IntTree` variables `t2` and `t3` refer to reference trees #2 and #3, respectively. The call of `t2.combineWith(t3)` will return a reference to the following new tree. Keep in mind that nodes numbered `1` are those that

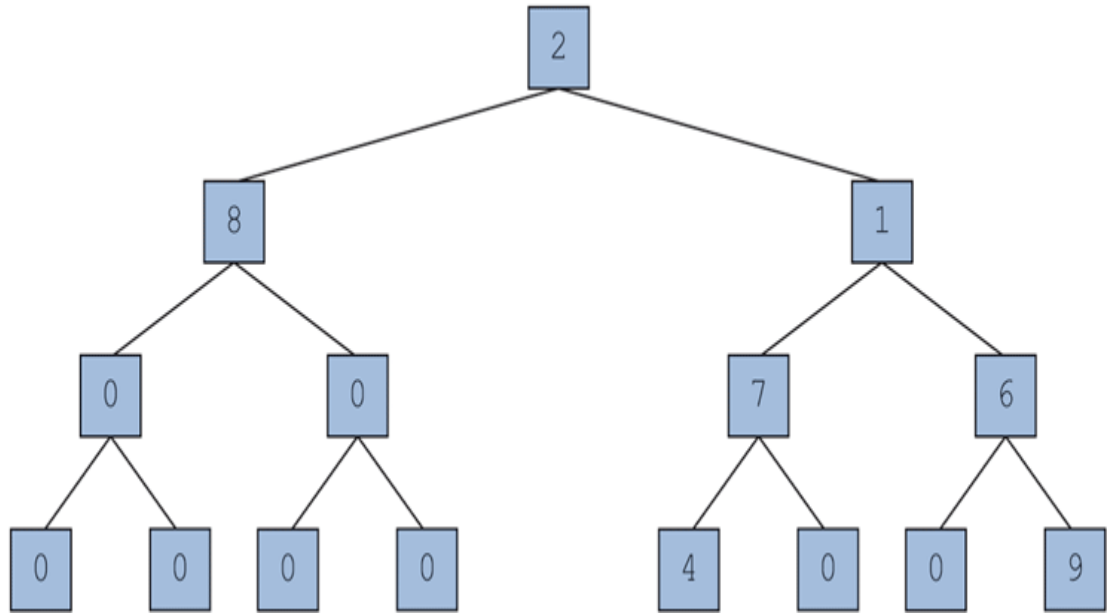
appear only in `t2`, nodes numbered `2` appear only in `t3`, and nodes numbered `3` appear in both.



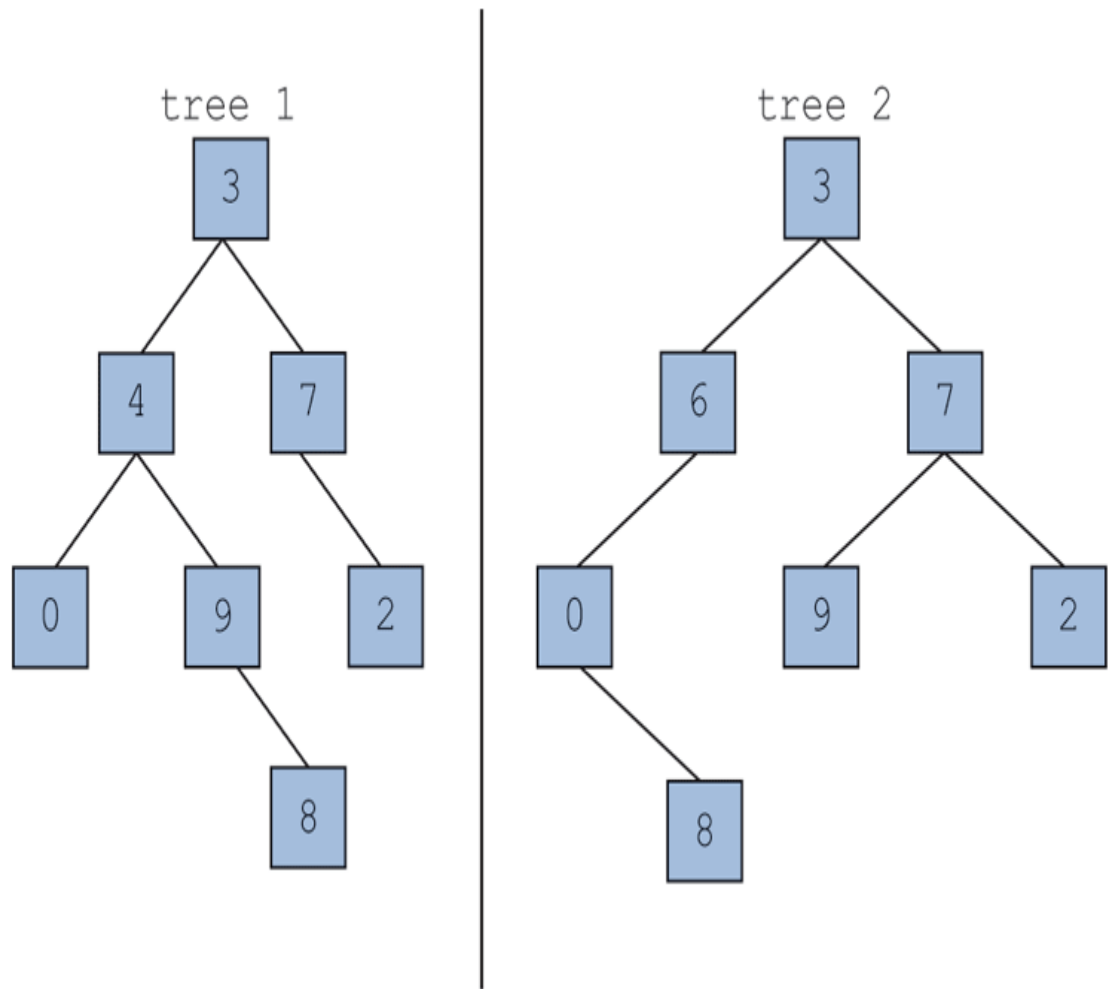
18. Write a method called `inOrderList` that returns a list containing the sequence of values obtained from an inorder traversal of your binary tree of integers. For example, if a variable `t` refers to reference tree #3, then the call `t.inOrderList()` should return the list `[8, 3, 7, 2, 1]`. If the tree is empty, your method should return an empty list.
19. Write a method called `evenLevels` that makes sure that all branches end on an even level. If a leaf node is on an odd level it should be removed from the tree. We will define the root as being on level `1`. For example, if a variable `t` refers to reference tree #2, then the call `t.evenLevels();` should change the tree's state to the following, removing the leaf value `0` because it is on an odd level:



20. Write a method called `makePerfect` that adds nodes until the binary tree is a *perfect* tree. A perfect binary tree is one where all leaves are at the same level. Another way of thinking of it is that you are adding dummy nodes to the tree until every path from the root to a leaf is the same length. A perfect tree's shape is exactly triangular and every branch node has exactly two children, and all of the leaves are at the same level. Each new node you add to the tree should store the value `0`. For example, if a variable `t` refers to reference tree #2, then the call `t.makePerfect();` should change the tree's state to the following:



21. Write a method called `matches` that returns a count of the number of nodes in one tree that match nodes in another tree. A match is defined as a pair of nodes that are in the same position in the two trees relative to their overall root and that store the same data. Consider, for example, the following trees:



The overall root of the two trees match (both are 3). The nodes at the top of the left subtrees of the overall root do not match (one is 4 and one is 6). The top of the right subtrees of the overall root match (both are 7). The next level of the tree has 2 matches for the nodes storing 0 and 2 (there are two nodes that each store 9 at this level, but they are in different positions relative to the overall root of the tree). The nodes at the lowest level both store 8, but they aren't a match because they are in different positions. Therefore, these two trees have

a total of 4 matches. Therefore the calls of `tree1.matches(tree2)` and `tree2.matches(tree1)` would each return 4.

Programming Projects

1. Implement a yes/no guessing game called 20 Questions using a binary tree. At the beginning of each round of the game, the human player thinks of an object. The computer tries to guess the object by asking a series of no more than 20 yes-or-no questions. Eventually the computer guesses what the object is. If this guess is correct, the computer wins; if not, the human player wins.

The computer keeps track of a binary tree with nodes representing questions and answers. A “question” node contains a left “yes” subtree and a right “no” subtree. An “answer” node is a leaf. This tree can be traversed to ask the human player questions until it reaches a leaf node, at which point the computer asks whether that answer is the correct one.

Initially the computer is not very intelligent, but it grows more intelligent with each game. If the computer’s answer guess is incorrect, the player must give it a new question to help it in future games. This is similar to the game found at the web site <http://animalgame.com/>. You should write methods to construct a tree, play a game, and save/load the tree’s state from a file.

2. Write a program that encodes and decodes Morse code files using a binary tree. Morse code is a system that encodes the 26 English letters and 10 numeric digits into sequences of

dots, dashes, and spaces. Your tree can store the encodings of each letter and number, going left for a dot and right for a dash. When your program reads a sequence of Morse code characters, it should traverse the tree in the appropriate direction for each character and output the letter or number it reaches at the end of each sequence.

3. Write a program that performs Huffman encoding and compression using a binary tree. Huffman coding is an algorithm devised by David A. Huffman of MIT in 1952 for compressing text data to make a file occupy a smaller number of bytes. The idea of Huffman coding is to abandon the rigid 8-bits-per-character requirement of ASCII encoding and use different-length binary encodings for different characters. The advantage of doing this is that if a character occurs frequently in the file, such as the letter *e*, it could be given a shorter encoding (fewer bits), making the file smaller.

Your program should contain methods to read an input text file, count its characters, and use these counts to build a Huffman tree for the characters by frequency. Use this Huffman tree to output an encoded and compressed version of the file. Also write a method to read and decompress this file later. You may want to use helper classes for reading and writing a file one bit at a time. You can find classes called

`BitInputStream` and `BitOutputStream` at our web site,

<http://buildingjavaprograms.com>.

4. The actual `Set` interface in the `java.util` package has several methods that are beyond those implemented by our search tree in this chapter. Write a version of `SearchTree<E>`

that adds some or all of these methods. The methods to add are the following (some headers are slightly modified; see the Java API Specification for descriptions of each method):

- `public void addAll(SearchTree<E> tree)`
- `public void clear()`
- `public boolean containsAll(SearchTree<E> tree)`
- `public boolean equals(Object o)`
- `public boolean isEmpty()`
- `public boolean remove(Object o)`
- `public void removeAll(SearchTree<E> tree)`
- `public void retainAll(SearchTree<E> tree)`
- `public Object[] toArray()`

5. Add an iterator to the search tree. Write a class called `SearchTreeIterator` that has the methods that follow for iterating over a binary tree. You will also need to modify the tree nodes to store parent references so that the iterator can properly walk “up” the tree as necessary. (See the Java API Specification for descriptions of each method.) Here are the methods:

- `public boolean hasNext()`
- `public E next()`
- `public void remove()`

6. Write a program that evaluates numeric expressions using a binary expression tree. A leaf node represents an operand (a

number). A branch node represents an operator; its two subtrees represent its operands.